



北方民族大学

本科毕业论文（设计）

题目： 基于 Vulkan 的实时光线追踪混合渲染器的设计与实现

学 院 名 称： 计算机科学与工程学院

专 业： 软件工程

学 生 姓 名： 向晨

学 号： 20221889

指导教师姓名： 韩强

企业指导老师：

论文提交时间：

北方民族大学毕业论文（设计）诚信承诺书

学生姓名	向晨	年级	2022 级
所学专业	软件工程	学号	20221889
所在学院	计算机科学与工程学院		

学生承诺

本人慎重承诺和声明：

我承诺在毕业论文（设计）过程中严格遵守学校有关规定，在指导教师的安排与指导下独立完成所规定的毕业论文（设计）工作，决不弄虚作假，不请别人代做毕业论文（设计）或抄袭别人的成果。所撰写的毕业论文或毕业设计是在指导老师的指导下自主完成，文中所有引文或引用数据、图表均注解并说明来源，本人愿意为由此引起的后果承担责任。

学生（签名）：

2024 年 05 月 日

教务处制

基于 Vulkan 的实时光线追踪混合渲染器的设计与实现

摘要

在现代计算机图形学中，传统的纯光栅化渲染难以表现出真实的全局光照与软阴影细节，而电影级的纯光线追踪又因为计算量过于庞大，难以满足交互式应用的实时帧率要求。为了在“画质”与“性能”之间找到最佳平衡，本文基于 Vulkan 1.3 图形引擎接口，设计并实现了一套高性能的实时光线追踪混合渲染引擎。

本文的核心工作与研究成果主要体现在以下三个方面：

首先，在底层架构设计上，本文构建了一套自动化的渲染调度系统（RenderGraph）。该系统能够自动分析渲染任务之间的依赖关系，智能接管复杂的显存分配与多线程同步工作。这不仅大幅提升了底层硬件的运行效率，也解决了复杂渲染管线中极其容易出现内存冲突问题。

其次，在混合渲染实现上，系统采用了“光栅化 + 硬件光线追踪”的混合架构。利用光栅化技术快速获取屏幕的几何信息，再结合光线追踪技术计算真实的阴影与物理反射。针对实时光追因采样率过低导致画面出现大量噪点的问题，本文优化了光源采样策略，引导探测射线优先计算高亮度的主要光源，从而在源头上大幅降低了画面的初始噪点。

最后，在画面降噪与优化上，本文引入了时空引导滤波降噪算法（SVGF）与时域抗锯齿（TAA）技术。系统通过巧妙地复用上一帧的历史画面数据，并结合当前帧的空间特征进行平滑处理，以极低的性能开销，将原本充满噪点的原始光影信号，还原成了平滑、细腻的真实物理画面。实验测试表明，本引擎能够在主流硬件上稳定运行，在呈现出极高真实感光影的同时，完全达到了实时交互的帧率标准。

关键词：Vulkan；三维渲染引擎；实时光线追踪；计算机图形学

Design and Implementation of a Real-time Ray Tracing Hybrid Renderer Based on Vulkan

Abstract

In modern computer graphics, traditional pure rasterization rendering struggles to present realistic global illumination and soft shadow details, while film-level pure ray tracing is difficult to meet the real-time frame rate requirements of interactive applications due to its excessive computational load. To find the optimal balance between "image quality" and "performance", this paper designs and implements a high-performance real-time ray tracing hybrid rendering engine based on the Vulkan 1.3 graphics engine interface.

The core work and research achievements of this paper are mainly reflected in the following three aspects:

Firstly, in the underlying architecture design, this paper constructs an automated rendering scheduling system (RenderGraph). This system can automatically analyze the dependency relationships between rendering tasks and intelligently take over complex video memory allocation and multi-thread synchronization. This not only greatly improves the operating efficiency of the underlying hardware, but also solves the memory conflict problem that is extremely prone to occur in complex rendering pipelines.

Secondly, in the implementation of hybrid rendering, the system adopts a hybrid architecture of "rasterization + hardware ray tracing". It uses rasterization technology to quickly obtain the geometric information of the screen, and then combines ray tracing technology to calculate realistic shadows and physical reflections. Aiming at the problem of a large number of noise points in the image caused by the low sampling rate of real-time ray tracing, this paper optimizes the light source sampling strategy, guiding the detection rays to prioritize the calculation of high-brightness main light sources, thereby greatly reducing the initial noise of the image from the source.

Finally, in terms of image denoising and optimization, this paper introduces the Spatiotemporal Variance-Guided Filtering (SVGF) and Temporal Anti-Aliasing (TAA)

technology. By cleverly reusing the historical image data of the previous frame and combining the spatial characteristics of the current frame for smoothing processing, the system restores the original noisy light and shadow signals into smooth and delicate real physical images with extremely low performance overhead. Experimental tests show that the engine can run stably on mainstream hardware, and while presenting highly realistic light and shadow, it fully meets the frame rate standard of real-time interaction..

Key words: Vulkan; 3D Rendering Engine; Real-Time Ray Tracing; Computer Graphics

目录

第一章 绪论	2
1.1 研究背景及意义	2
1.2 研究现状	3
1.3 本文研究内容	4
1.4 论文总体结构	4
第二章 实时混合渲染器相关技术	6
2.1 实时混合渲染技术	6
2.1.1 光栅化	6
2.1.2 光线追踪	7
2.2 基于物理的渲染	8
2.2.1 微面元理论 (Microfacet Theory)	8
2.2.2 能量守恒 (Energy Conservation)	8
2.2.3 基于物理的 BRDF (Physically Based BRDF)	8
2.3 实时渲染管线	10
2.3.1 前向渲染管线	10
2.3.2 延迟渲染管线	11
2.4 图形 API	11
2.4.1 OpenGL 及其他图形 API	12
2.4.2 Vulkan	12
2.5 本章小结	14
第三章 需求分析	15
3.1 可行性分析	15
3.2 系统业务流程分析	15
3.3 系统功能性需求分析	16
3.3.1 用户功能需求	16

3.4 系统非功能性需求分析	19
3.4.1 性能和响应时间:	19
3.4.2 安全性与稳定性	20
3.4.3 可用性与可靠性	20
3.4.4 易用性	20
3.4.5 可维护性与可扩展性	20
3.5 系统核心类设计	21
3.5.1 全局管理与框架类设计	21
3.5.2 资源管理与场景组织类设计	21
3.5.3 渲染调度引擎核心类设计	22
3.6 本章小结	23
第四章 概要设计	24
4.1 引擎的架构设计	24
4.1.1 Walnut.....	24
4.1.2 RenderGraph.....	25
4.1.3 本引擎架构设计	26
4.2 功能模块划分	27
4.2.1 图形硬件后端模块	27
4.2.2 资源管理模块	27
4.2.3 场景管理模块	27
4.2.4 渲染调度模块	27
4.2.5 混合渲染管线模块	28
4.2.6 交互调试模块	28
4.3 混合渲染管线流程设计	28
4.3.1 几何特征采集	29
4.3.2 物理光影计算	29
4.3.3 时空信号重建	29
4.3.4 光照合成与后期处理	30

4.4 本章小结	30
第五章 详细设计	31
5.1 图形硬件后端模块	31
5.1.1 基于 volk 的动态函数指针加载机制	31
5.1.2 物理特性按需注入机制	33
5.1.3 交换链 (Swapchain) 管理与异常恢复	34
5.2 资源管理模块	34
5.2.1 静态架构：无绑定(Bindless)资源池设计	35
5.2.2 动态加载：基于引用计数的异步流转机制	35
5.2.3 光源预处理：蒙特卡洛积分 CDF 硬件同步	36
5.3 场景管理模块	38
5.3.1 混合光追加速结构(TLAS/BLAS)动态构建机制	38
5.3.2 基于八叉树的视锥体剔除策略	39
5.3.3 时域抖动注入与历史矩阵追踪系统	39
5.4 渲染调度模块	40
5.4.1 任务编排与资源依赖的声明式建模	41
5.4.2 自动同步屏障推导与状态转换机制	41
5.4.3 异构计算任务的执行上下文封装	43
5.5 混合渲染管线模块	43
5.5.1 基于光栅化的几何特征提取（G-Buffer）	43
5.5.2 单样本（1-SPP）光线追踪探测	44
5.5.3 时空方差引导滤波（SVGF）降噪实现	46
5.5.4 光照合成与后期处理管线	46
5.6 交互调试模块	48
5.6.1 应用生命周期与事件驱动机制	48
5.6.2 非侵入式参数反控与闭环调优	49
5.6.3 图形界面后端与性能可视化	49
5.7 本章小结	50

第六章 系统实现与测试	51
6.1 系统运行环境	51
6.2 系统功能实现	51
6.2.1 交互式调试界面展示	51
6.2.2 G-Buffer 几何特征可视化.....	52
6.2.3 光线追踪与降噪算法对比	52
6.2.4 后期处理与最终画面输出	53
6.3 系统测试	53
6.4 本章小结	54
第七章 总结与展望	55
7.1 全文工作总结	55
7.2 研究不足与未来展望	55
参考文献	57
致谢	59

第一章 绪论

1.1 研究背景及意义

实时渲染是专注于制作和分析实时图像的计算机图形学的子领域，其旨在极短的时间内完成从三维几何数据到二维图像的计算与生成。在典型的实时交互应用中，系统需要持续捕获用户的输入，更新虚拟环境状态，并以较高的刷新率输出连续的图像序列。通常，帧率（FPS）^①大于 30 的渲染被称为实时渲染，10FPS 左右的我们称为可交互渲染，再慢的就是离线渲染。实时渲染技术目前广泛应用在强交互的 3D 游戏和带 3D 的应用中，例如，游戏，VR/AR，虚拟仿真等，有巨大的应用前景和商业价值。

在技术路线上，实时渲染主要分为光栅化（Rasterization）与光线追踪（Ray Tracing）两种基础架构。光栅化技术基于多边形投影原理，其算法流程高度契合 GPU 的单指令多数据流（SIMD）并行计算架构，能够以极高的效率处理大规模几何数据。然而，光栅化本质上是一种基于局部信息的计算模型，缺乏对场景全局几何拓扑的直接感知能力。因此，在处理全局光照（Global Illumination）、软阴影及物理反射时，传统光栅化管线主要依赖屏幕空间近似算法（如屏幕空间反射 SSR、屏幕空间环境光遮蔽 SSAO）或预计算技术（如阴影贴图、光照探针）。这些启发式近似方法在处理复杂遮挡关系与动态场景时，往往会产生走样（Aliasing）及物理失真问题。

相对而言，光线追踪技术通过在三维空间中模拟光线的物理传播、散射与折射路径，能够更准确地求解渲染方程，从而生成具备物理真实感的光影效果。但由于光线在场景中弹射路径具有较高的随机性，导致内存访问呈现非连续特征，难以充分利用 GPU 的并行算力，因此长期被限制在离线渲染领域。

随着 GPU 硬件算力的提升以及专用硬件光线追踪加速单元（如 RT Core）的引入，实时光线追踪技术逐渐具备了工程可行性。为平衡渲染质量与计算开销，混合渲染架构（Hybrid Rendering Architecture）成为当前的主流发展方向。该架构将光栅化在初级可见性（Primary Visibility）计算上的高效率，与光线追踪在次级光线（如反射、阴影）求解上的准确性相结合，有效弥补了单一管线的不足，是目前图形学界重点研究的核心课题。

^① 通常会使用每秒显示的帧数（frames per second, FPS）或者赫兹（Hertz, Hz）来衡量图像显示的速率。

1.2 研究现状

图形渲染管线及底层硬件经历了长期的演进过程。20 世纪 90 年代末，图形硬件主要采用固定渲染管线（Fixed-Function Pipeline），开发者仅能通过配置预设状态来控制渲染输出。2001 年起，可编程着色器（Programmable Shaders）的引入使开发者能够自定义顶点变换与像素着色逻辑。至 2006 年，统一着色器架构（Unified Shader Architecture）的提出，打破了不同着色阶段计算资源的物理隔离，为现代通用 GPU 计算奠定了基础。2018 年是实时渲染的分水岭，Microsoft 发布了 DXR（DirectX Raytracing）框架，并且 NVIDIA 在同年推出了基于 Turing 架构的 RTX 平台，让实时光线追踪成为可能。

在底层图形应用程序接口（Graphics API）方面，传统 API（如 OpenGL 与 DirectX 11）由于设计于单核 CPU 时代，采用了高度封装的全局状态机模型。在这种架构下，内存管理与线程同步操作由图形驱动程序隐式处理，导致在处理复杂场景时产生较高的 CPU 驱动开销，限制了多核处理器的并发能力。为此，工业界推动了向显式图形 API（Explicit Graphics API）的范式转移，如 Vulkan 与 DirectX 12。此类 API 将显存的分配与释放、多线程命令缓冲区的录制以及管线同步屏障（Pipeline Barriers）的控制权显式移交至应用层，显著降低了驱动开销，提升了硬件算力的利用率。

渲染器是游戏引擎中核心的子系统之一，它负责图形绘制，接收渲染命令和资源数据（顶点、纹理、Shader 等），并调用 OpenGL/D3D/Vulkan 等图形 API，将游戏世界中的数据（模型、纹理、光照、摄像机等）转换为最终呈现在屏幕上的图像。最早可追溯至 id Software 开发的游戏《德军总部 3D（Wolfenstein 3D）》《毁灭战士（DOOM）》取得了巨大成功，其主要的开发者约翰卡马克将可复用的底层代码进行剥离，创造了游戏引擎这一概念。DOOM 引擎是第一个用于商业授权的引擎，后来开发的 Quake 引擎是真正的 3D 引擎，人们普遍认为游戏《雷神之锤》带来了独立 3D 显卡的革命。商业游戏引擎市场主要由 Unreal 与 Unity 占据主导地位，开源领域亦涌现出 Orge, Godot 等引擎，当然也还有专门的渲染引擎，比如 V-Ray, OctaneRender 等，给许多引擎和渲染开发者提供了优秀的学习资料，促进了游戏引擎，渲染领域的发展。

在渲染引擎架构层面，随着渲染管线复杂度的增加，传统的面向对象绘制指令流逐渐难以维护。现代工业级渲染引擎广泛引入了基于数据驱动的渲染图架构 RenderGraph。

该架构通过构建有向无环图（DAG），在编译期自动推导各渲染通道（Render Pass）之间的数据依赖关系，从而实现显存的动态复用与同步屏障的自动插入，是目前处理复杂混合渲染管线状态调度的标准方案。

此外，在实时光线追踪的工程实践中，受限于实时计算的性能预算，系统通常仅能采用极低的采样率（如单像素单样本，1-SPP），导致原始渲染结果存在严重的蒙特卡洛噪声。因此，时空重建与降噪技术（Spatiotemporal Reconstruction and Denoising）成为当前的研究热点。以时空方差引导滤波（SVGF）为代表的算法，通过提取历史帧的时间相关性信息并结合当前帧的空间几何特征，能够有效平滑高频噪声。降噪算法的执行效率与重建质量，已成为决定混合渲染管线最终视觉表现的关键因素。

1.3 本文研究内容

本文基于 Vulkan 1.3 规范设计并实现了一个三维实时混合渲染引擎，主要研究内容包括以下几个方面。

- （1）研究 Vulkan 1.3 渲染管线，利用动态渲染等新特性优化引擎底层架构，探索资源管理流程。
- （2）基于有向无环图的渲染调度系统设计：设计并实现 RenderGraph 渲染管线调度模块。通过分析渲染通道间的资源读写依赖，实现显存资源的生命周期管理与管线同步屏障的智能推导，解决复杂渲染管线中的状态同步与资源别名（Aliasing）问题。
- （3）构建混合渲染管线，设计基于延迟渲染（DeferredRendering）的混合光照管线。利用光栅化技术提取几何缓冲（G-Buffer），并结合 Vulkan 硬件光线追踪扩展，实现高质量的软阴影与反射效果。
- （4）针对低采样率硬件光线追踪带来的噪声问题，在管线后端集成 SVGF 时空滤波降噪算法。利用运动矢量复用历史帧数据，结合基于几何属性的空域边缘保留滤波，实现渲染图像质量与运行帧率的平衡。

1.4 论文总体结构

以下是对本论文各个章节的概要：

第一章 绪论：阐述实时混合渲染技术的研究背景及意义，分析图形硬件、API 及渲染算法的发展现状，并简述本文的主要工作。

第二章 理论基础：介绍分析 光栅化、光线追踪、PBR 物理模型和图形 API。

第三章 需求分析：从可行性、业务流程、功能与非功能指标等方面详细阐述引擎需求。

第四章 概要设计：分析现有技术，引出渲染器分层架构，分析设计混合渲染管线。

第五章 详细设计：详细阐述第四章的模块分层设计和渲染管线。

第六章 测试：通过具体的渲染场景对引擎的功能进行验证，并结合性能分析工具，对系统的显存占用、帧率表现及各渲染通道的时间开销进行量化评估。

第七章 总结与展望：对本文的主要工作与研究成果进行总结，分析当前引擎架构存在的局限性，并对未来的技术优化方向进行展望。

第二章 实时混合渲染器相关技术

本章将对构建实时渲染器所涉及的相关技术进行介绍，包括渲染技术，基于物理的渲染，混合渲染管线和图形 API 介绍。

2.1 实时混合渲染技术

渲染是指通过软件算法将包含几何、视点、纹理和照明等信息的三维（或二维）模型最终转换为二维图像的技术，其过程可类比为物理相机的成像原理，即将三维空间信息投影至二维平面之上。可以看出渲染的数据源主要包含模型，相机和光源，而如何处理这些数据主要有下面两种方法。

2.1.1 光栅化

光栅化是现代实时渲染中最经典、应用最广泛的技术之一。光栅化可以理解为将 3D 空间中的三角形投射到 2D 屏幕^②上的一个过程。其外层循环遍历场景中的几何图元（三角形），内层循环寻找覆盖该图元的屏幕像素。

在数学层面上，光栅化首先通过一系列仿射变换（仿射矩阵乘法），将三维空间中的顶点坐标连续映射至二维的标准化设备坐标系（NDC）。随后，算法进入核心的离散化阶段：利用边方程（Edge Equations）或重心坐标（Barycentric Coordinates）算法，对屏幕上的像素网格进行内外测试（Inside-Outside Test），从而确定哪些像素中心落入了当前三角形的投影包围盒内。

针对可见性问题，光栅化依赖于深度缓冲（Z-Buffer）算法。它在内存中维护一张与屏幕分辨率一致的二维浮点数组，当多个三角形投影到同一像素时，算法通过比较当前片段的插值深度 Z_{new} 与缓冲区中的历史深度 Z_{old} ，仅保留离观察者最近的片段。

光栅化的优势是契合 GPU 的并行计算架构，能很好处理初级可见性（Primary Visibility）的问题。光栅化算法在着色阶段（Shading Stage）本质上是一种严格的局部计算模型。在现代图形硬件的片段着色器（Fragment Shader）中，大规模并行处理的各个计算单元仅能访问当前片段的局部属性（如插值后的法线、纹理坐标、材质参数）以及全局一致变量（Uniforms，如直接光源方向），缺乏对场景全局几何拓扑信息的感知能力。由于无法直接探测场景中的其他几何体，光栅化架构下的阴影、环境遮蔽与反射通

^② 在图形学中我们可以把它抽象为一个二维数组，其中二维数组中的每个元素是像素（pixel）

常依赖于启发式近似(Heuristics)或预处理技术(如 Shadow Maps、Environment Maps)，这些技术在处理动态场景与复杂遮挡关系时往往存在走样(Aliasing)或精度缺失的问题。这对物理光照模型做了简化，导致光影效果相对简单，无法完全模拟真实世界中的复杂光照现象。

2.1.2 光线追踪

与光栅化的投射思路不同，采用基于采样(Sampling)的策略。其外层循环遍历屏幕上的二维像素，内层循环则在三维场景中寻找与该像素视线相交的几何图元。

光线追踪采取了一种逆向模拟光路物理传播的思路，从摄像机发射光线，与物体表面相交后，根据材质的粗糙度、折射率继续反弹，直至与光源相交或达到最大迭代深度。

在算法表达上，光线追踪通过逆向模拟光路传播，从观察点向空间发射射线。射线的数学参数方程定义为：

$$P(t) = O + t \cdot D \quad (2-1)$$

其中 O 为射线原点， D 为标准化的方向向量， t 为光线传播的标量距离。算法的核心算力被消耗在求解射线 $P(t)$ 与场景中所有三角形平面的代数交点上。

为了避免 $O(N)$ 级别的暴力求交导致的性能崩溃，现代光线追踪算法必须依赖空间加速结构(Acceleration Structure)。通常采用层次包围盒(Bounding Volume Hierarchy, BVH)或 KD 树，将场景几何体在三维空间中进行层次化切分，使得单根光线求交的时间复杂度从线性的 $O(N)$ 骤降至对数级别的 $O(\log N)$ 。当射线与场景中的几何体的空间加速结构相交时，可以逆推出该点的入射光线方向，根据交点处的材质属性，生成反射光线、折射光线以及阴影光线。通过逐层递归计算，最终将各层的光照结果累积起来，得到像素的颜色。

相较于光栅化，光线追踪能够更好地还原物理光照效果，在全局光照、镜面反射、透明折射、软阴影等细节表现上更具优势，画面真实感更强，能够呈现出接近现实世界的视觉效果。但由于采样的随机性，无法适应 GPU 的 SIMD 并行架构，需要大量光线迭代求交运算，整体计算成本极高，传统纯光线追踪渲染难以满足高帧率实时交互的需求，通常用于离线渲染、影视特效等对实时性要求较低的场景。直到近几年，NVIDIA、AMD 等硬件厂商在 GPU 中加入了 RT Core（硬件光追核心），才让实时光线追踪成为可能。

2.2 基于物理的渲染

在早期的图形学中，我们广泛使用的是 Phong 或 Blinn-Phong 光照模型。这些模型被称为经验模型，对光照的一种模拟，通过实践总结出简化的方法，简化了真实的光照计算。这种传统的模型的参数有许多随意性和技巧（tricks），在不同的光照条件、引擎下是不通用的。为了让材质的工作流统一，PBR 诞生了。PBR 并不是一个单一的公式，而是一套基于真实世界光学物理法则的渲染规范。其核心理论有三个：微面元理论、能量守恒 以及 基于物理的 BRDF。

2.2.1 微面元理论 (Microfacet Theory)

该理论认为，任何在宏观视觉上平滑的表面，在微观尺度下都是由无数个朝向各异的微小平面（微面元）构成的。如图 2.1 所示，材质的粗糙度（Roughness）直接决定了这些微面元法线的分布规律：表面越光滑，微面元的朝向越高度一致，反射光线越平行集中，从而形成明亮且锐利的镜面高光；表面越粗糙，微面元的朝向越杂乱无章，反射光线被散射到各个方向，从而形成宽广而黯淡的高光。

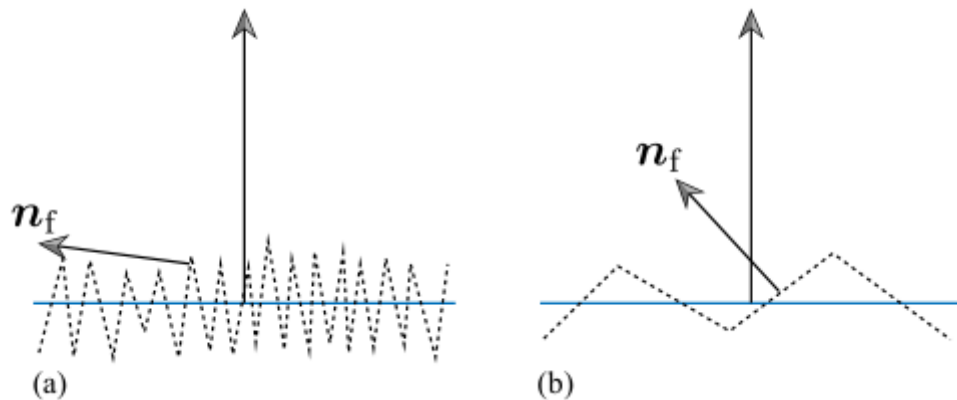


图 2.1 微表面模型原理图。(a) 粗糙表面；(b) 光滑表面

2.2.2 能量守恒 (Energy Conservation)

基于物理学的能量守恒要求：出射光的总能量不能超过入射光的总能量。一束光打在介质表面后，必然分为两部分：立刻从表面弹射的反射光（Specular），以及穿透表面进入内部被吸收或散射出的漫反射光（Diffuse）。表面直接反射的光越强，能够进入内部参与漫反射的光就越少，这样来维持物理上的真实感。

2.2.3 基于物理的 BRDF (Physically Based BRDF)

为了用数学语言精确计算上述物理现象，PBR 采用了基于物理的双向反射分布函

数（BRDF）。现代 PBR 框架通常采用 Cook-Torrance BRDF 模型，它将光照计算科学地拆分为漫反射项与镜面反射项。通项公式如下：

$$f_r(l, v) = k_d f_{diffuse} + k_s f_{specular} \quad (2-1)$$

式中， k_d 和 k_s 分别是漫反射和镜面反射的能量权重系数， $f_{lambert}$ 是 Lambert 漫反射模型，其计算表达式为：

$$f_{lambert} = \frac{c}{\pi} \quad (2-2)$$

c 表示表面的基础反照率（Albedo），即表面颜色。除以 π 是为了保证漫反射在半球空间积分下的能量守恒。

$f_{cook-torrance}$ 则为 Cook-Torrance 镜面反射模型，专注于模拟微面元在物理层面的高光贡献如下：

$$f_{cook-torrance} = \frac{D(h)F(v, h)G(l, v, h)}{4(n \cdot l)(n \cdot v)} \quad (2-3)$$

式中， n 表示物体表面法线， v 表示观察方向， l 表示光源入射方向， h 表示视线与光线的半程向量。 $D(h)$ 是法线分布函数， $F(v, h)$ 是菲涅尔项， $G(l, v, h)$ 是几何遮挡函数。这三个函数分别用来描述微表面的粗糙度分布、菲涅尔效应和几何遮挡效应。

其中，针对镜面反射的计算依赖于三个物理函数（俗称 DFG 项）：

法线分布函数 $D(h)$ 从统计学的角度近似地表示了与半程向量 h 取向一致的微表面的比率，该项直接决定了高光的形状。本文采用 Trowbridge-Reitz GGX 模型来进行估算，见式 2-4

$$D(\vec{n}, \vec{h}, \alpha) = \frac{\alpha^2}{\pi \left((\vec{n} \cdot \vec{h})^2 (\alpha^2 - 1) + 1 \right)^2} \quad (2-4)$$

GGX 模型相比于传统的 Blinn-Phong 分布，具备更平滑的拖尾（Tail）过渡特征，能够渲染出更具层次感的物理高光边界。公式中的 α 为材质的表面粗糙度系数。

几何函数 (G 项): 计算微面元之间在微观层面的相互遮挡与阴影衰减。微观层面的凹凸不平会导致光线在射入和反射时被自身结构遮挡。为此, 系统采用了分离式的 Smith-Schlick-GGX 几何近似模型, 将遮挡项分解为视线方向与光照方向两个独立子项的连乘:

$$G(l, v, h) = G_{sub}(l) \cdot G_{sub}(v) \quad (2-5)$$

单一方向分量 $G_{sub}(w)$ 的计算依赖于基于粗糙度重映射的系数 k :

$$G_{sub}(w) = \frac{n \cdot w}{(n \cdot w)(1 - k) + k} \quad (2-6)$$

其中, 变量 w 是一个代指方向向量的通用占位符。在实际求解时, 会将视线方向 v 与光源方向 l 分别代入 w 进行两次求值。对于实时直接光照计算, 系数 k 通常被重映射为

$$k = \frac{(\alpha + 1)^2}{8} \quad (2-7)$$

以避免极度粗糙表面处产生异常的暗部遮挡。

菲涅尔方程 (F 项): 描述现实世界中极为普遍的现象——当观察视线越趋近于平行表面 (掠射角) 时, 表面的反射率越强。

$$F(\vec{h}, \vec{v}, F_0) = F_0 + (1 - F_0) \left(1 - (\vec{h} \cdot \vec{v})\right)^5 \quad (2-8)$$

变量 F_0 代表材质在垂直视角下的基础反射率。该近似方案在保留物理特性的同时, 极大地削减了 GPU 的着色指令开销。

2.3 实时渲染管线

在了解了光栅化和光线追踪的基本原理后, 我们需要知道这些理论是如何在现代游戏引擎中被组织并执行的。这就引出了渲染管线 (Rendering Pipeline) 的概念。

渲染管线本质上是一套流水线化的数据处理流程: CPU 提交的 3D 顶点数据 (Vertices), 经过几何变换、光栅化、片段着色, 最终转化为屏幕上的 2D 像素 (Pixels)。随着人们对渲染效果的要求不断提高, 渲染管线也经历了从前向渲染到延迟渲染的变革, 两者的主要区别根据光照计算时机的不同。

2.3.1 前向渲染管线

前向渲染是最基础、最直观的渲染方式。目前市面上几乎所有的移动端轻量级游戏, 以及原生的 OpenGL/Vulkan 教学案例, 默认采用的都是前向管线。前向渲染的核心逻辑

辑是：渲染器会遍历场景中的每个物体，对于物体上的每个可见像素，计算场景中所有光源对该像素的光照贡献，最后输出到颜色缓冲区。其可见性测试（深度测试）与着色计算（Shading）是同一阶段耦合进行的。但当多个物体在屏幕上重叠时，一个像素可能会被多个重叠的三角形覆盖。而这些被遮挡的物体碎片仍然可能执行复杂的着色计算。这种多余的计算在场景中有多个光源时会产生极大浪费，适合光源较少的情景。前向渲染的优点是它天然支持半透明物体（Translucency）渲染，并且能够兼容硬件级的多重采样抗锯齿（MSAA）。

2.3.2 延迟渲染管线

延迟渲染管线是现代的高效渲染管线技术，适用于光源较多、场景复杂的应用。将可见性测试与着色计算完全解耦，主要可以分为两个阶段。

首先是几何处理阶段：渲染器不进行任何光照计算，只遍历场景中的物体，将场景中每个像素的几何信息和材质属性（如法线 Normal、反照率 Albedo、粗糙度 Roughness、深度 Depth）提取出来，分别渲染到几张屏幕大小的纹理中，这组纹理被统称为 G-Buffer。由于有深度测试（Z-Test），G-Buffer 中最终留下的一定是离摄像机最近的、不会被遮挡的像素数据。

然后是光照处理阶段：在这一步，我们不考虑 3D 几何体，只在屏幕上绘制一个全屏的 2D 矩形。片段着色器会读取 G-Buffer 中每个像素的属性，然后结合场景中的光源进行光照计算。现在无论场景有多复杂，重叠了多少层物体，每个屏幕像素永远只计算一次光照，避免了光照的 Overdraw 问题。

延迟渲染减少了光照计算的开销，但是缺点也很明显。G-Buffer 需要在极高的分辨率下存储大量的浮点数信息，会占用大量的显存带宽（Bandwidth）。由于 G-Buffer 中存储的是离散的属性数据，传统的 MSAA 无法直接作用于光照结果，需要使用 TAA（时域抗锯齿）等后处理抗锯齿方案。并且 G-Buffer 每个像素只能存储一层数据，无法记录半透明物体背后的信息。

2.4 图形 API

图形应用程序接口（Graphics Application Programming Interface，简称图形 API），是连接应用程序与图形硬件（GPU）的软件接口，负责将应用程序中的渲染指令（如几何绘制、光照计算、纹理采样等）转换为 GPU 能够识别和执行的指令，是构建实时混

合渲染器的重要工程基础。目前业界主流的图形 API 主要有 OpenGL、DirectX、Metal 和 Vulkan，各自具有不同的特点和适用场景，需开发者根据渲染器的开发需求和目标平台进行权衡。

2.4.1 OpenGL 及其他图形 API

OpenGL 是图形学领域历史最悠久、应用最广泛的跨平台图形 API 之一。于 1992 年由 Silicon Graphics, Inc. (SGI) 开发，在 2006 年转为由非营利组织 Khronos Group 管理。OpenGL 采用的是基于全局状态机的设计模型。其开放性强、生态成熟，拥有丰富的函数库和工具支持，API 易于使用，降低图形编程的门槛——开发者不需要关心内存是如何分配的，也不需要管理底层的多线程同步。但也因为大部分底层操作由 OpenGL 自己处理，导致每次绘制调用（Draw Call）前都需要进行繁重的状态合法性校验，有巨大的性能瓶颈。在 2017 年发布 OpenGL4.6 后，Khronos Group 转而全力开发 Vulkan 图形 API，OpenGL 已经停止更新。

DirectX 是由微软公司开发的图形 API，仅支持 Windows 和 Xbox 系列游戏主机平台，是 Windows 平台下游戏开发的主流 API。DirectX 经过多代迭代，目前最新版本为 DirectX 12。DirectX 11 提供了极佳的稳定性和完善的工具链，至今仍有大量游戏采用。DirectX 12 顺应了底层优化的趋势，引入了显式内存管理和多线程指令提交，在高端实时渲染、3A 游戏开发中具有显著优势。

Metal 是苹果公司在 2014 年苹果为 macOS, iOS 等系列产品定制的图形和计算 API，优势在于能最大程度地发挥苹果自研芯片的内存架构优势，提供了低开销（Low-overhead）和直接访问 GPU 的能力。

2.4.2 Vulkan

Vulkan 是 Khronos Group 推出的新一代跨平台、低开销（Low-overhead）图形 API。与 OpenGL 传统的隐式全局状态机设计不同，Vulkan 采用了显式（Explicit）的架构理念。如图 1.1 所示，Vulkan 提供一个更精简的驱动层，应用层获得更多控制权则能更精细化的控制渲染流程，给用户提供了更加底层的 API 调用，这种设计增加了 Vulkan 的门槛，但换来了巨大的性能提升。

Vulkan 提供了完善的跨平台光线追踪扩展（如 VK_KHR_ray_tracing_pipeline），让开发者能在统一的硬件抽象层中启用硬件加速的光线追踪通道。

Vulkan 提供了如下的现代特性：

1. 指令队列。Vulkan 采用显式的指令提交机制。GPU 的硬件能力被抽象为不同的指令队列（Command Queues），通常包括图形队列（Graphics）、计算队列（Compute）和传输队列（Transfer）。开发者首先将渲染指令记录在命令缓冲（Command Buffers）中，然后通过队列提交（Queue Submit）操作将指令送往 GPU 执行。这种架构允许引擎在多个 CPU 核心上并行构建指令，并支持异步计算队列与图形队列并发运行，从而最大化硬件的吞吐量。
2. 同步屏障。为了确保 GPU 异构核心之间的数据一致性，Vulkan 引入了同步屏障（Pipeline Barriers）。开发者必须显式地定义指令之间的执行依赖关系和内存访问控制。通过设置源阶段（Source Stage）和目标阶段（Destination Stage）以及对应的访问掩码（Access Mask），同步屏障能够有效解决指令流中的“写后读”（RAW）等数据冒险问题。在复杂的 RenderGraph 调度中，同步屏障是保证 G-Buffer 写入与光线追踪读取之间时序正确的关键手段。
3. 内存布局转换。Vulkan 中的图像资源（Images）在不同的渲染阶段需要处于特定的内存布局（Memory Layouts）中，以适配硬件的性能优化或特定的访问需求。例如，一张纹理在作为渲染目标时需处于 `COLOR_ATTACHMENT_OPTIMAL` 布局，而在着色器中被读取时则需转换为 `SHADER_READ_ONLY_OPTIMAL` 布局。这种转换通常通过内存屏障显式触发，驱动程序据此执行必要的缓存刷新（Cache Flush）或数据重排操作，确保资源在不同阶段间的正确衔接。

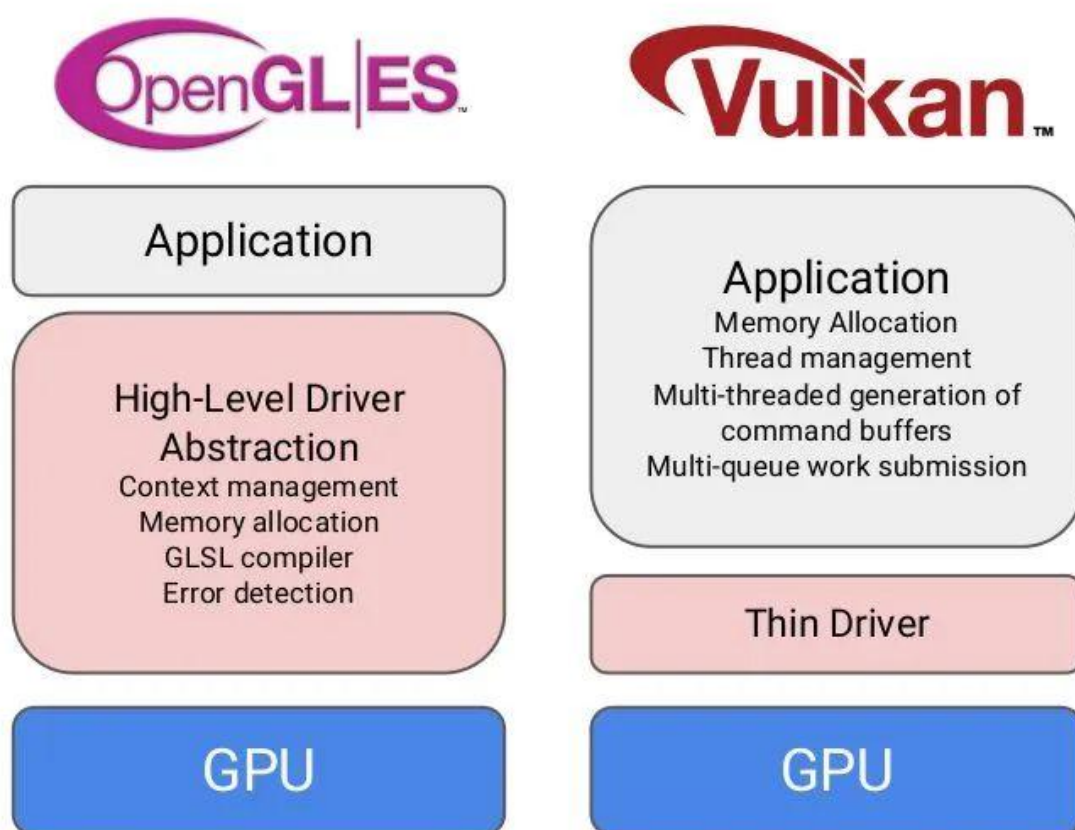


图 2.2 OpenGL Vulkan API 对比图

2.5 本章小结

本章系统性地梳理了构建实时混合渲染器所需的核心基础理论与底层工程技术。首先，对比分析了传统光栅化与光线追踪在处理可见性与全局光照时的优势与局限，确立了混合渲染路线的必要性；其次，深入探讨了基于物理的渲染（PBR）体系，涵盖微面元理论、能量守恒定律以及 Cook-Torrance BRDF 的数学推导，为材质系统的真实感奠定了光学与数学基础；随后，介绍了实时渲染管线从前向渲染向延迟渲染架构的演进过程与核心解耦思想；最后，横向对比了主流图形 API 的生态现状，并着重剖析了 Vulkan API 显式架构、指令队列与同步屏障等现代特性。本章所述的各项图形学理论与现代 API 规范，为后续章节中自研混合渲染引擎的架构设计与具体算法落地提供了坚实的理论指导与技术支持。

第三章 需求分析

本章将从软件工程的角度对混合渲染引擎进行需求分析。通过对可行性、业务流程及功能/非功能需求的梳理,确立系统在图形表现、调度效率及用户交互方面的具体基准。

3.1 可行性分析

（1）经济可行性：

本项目完全基于开源生态构建，核心开发环境与依赖库均具有较高的经济效益：

- 开发成本：Vulkan SDK、CMake、编译器（MSVC/GCC）以及图形驱动开发工具均可免费获取，无需支付商业许可证费用。
- 开源库集成：集成了 GLFW、Dear ImGui、GLM 及 VMA 等开源领域流行的库，本身的开源证书限制较小，不要求支付版权费用。
- 硬件成本：随着实时光线追踪显卡（RTX 系列）的普及，主流消费级硬件即可作为研发平台，研发设备投入处于合理区间

（2）技术可行性：

- 标准支持：系统基于 Vulkan 1.3 标准及 KHR Ray Tracing 扩展，利用 C++20 现代特性实现。当前硬件架构已提供成熟的硬加速求交单元（RT Core）。
- 知识获取：Vulkan 的官方文档、开源引擎的源码以及学术论文为技术实现提供了丰富的参考资料。
- 开发工具：业界主流的集成开发环境（如 Visual Studio）提供了完善的代码编写支持，各种开源库也能支持引擎的资源加载，图像界面，输入输出等功能。利用 Vulkan 验证层（Validation Layers）及 RenderDoc 调试工具，方便调试。

（3）操作可行性：

交互直观性：系统集成可视化 UI 调试面板，用户可实时通过 UI 界面，查看 Debug 信息，加载模型，移动光源，设置调试参数，以快速验证渲染的效果。

- 交互控制：参考成熟的游戏引擎的相机，交互的实现，降低用户的操作门槛。

3.2 系统业务流程分析

本混合渲染器的业务流程如下所述：

1. 运行应用：启动渲染器，初始化 Vulkan 上下文与图形驱动环境，加载窗口界面。

2. 场景加载与解析：用户通过 UI 界面选择 3D 模型和天空盒，引擎会后台自动进行资源加载与解析。

3. 渲染节点构建：系统根据用户选定的渲染路径（光栅化、混合路径或光追路径）自动构建 RenderGraph 拓扑图，分配资源和 Pass。

4. 交互调优：用户通过菜单栏切换不同的渲染模块，实时调整光线方向，SVGF、TAA 开关，通过键盘鼠标调整相机位置和远近。

5. 数据反馈与可视化：系统在主界面生成图像，同时支持用户切换查看 G-Buffer 法线、深度等中间渲染视图，方便查看每个阶段的效果，以及耗时统计等 Debug 信息。

3.3 系统功能性需求分析

混合渲染器在设计时以图形开发者和环境艺术家为核心用户，基于其在场景搭建、效果调优及性能监控中的需求进行功能分析。以下 3.3.1 小节详细描述了用户的具体功能需求。

3.3.1 用户功能需求

用户作为系统的核心操作者，应具有如下功能：

1. 场景与资产管理：该模块负责对渲染场景中的 3D 资产及其空间属性进行直观管理：
2. 资源浏览与加载：系统自动扫描指定目录下的模型与环境贴图。用户通过资源浏览器（Content Browser）可在多个 GLTF 模型间切换加载，并支持一键更换 HDR 环境天空盒。
3. 场景层级管理：用户通过层级面板（Hierarchy）实时查看当前场景中的所有实体对象，支持选中特定实例以进行后续操作，并提供安全移除选定实体的功能。
4. 变换编辑：支持对选中实体的平移（Position）、旋转（Rotation）和缩放（Scale）参数进行实时数值调节，并同步更新底层的硬件加速结构（AS）。

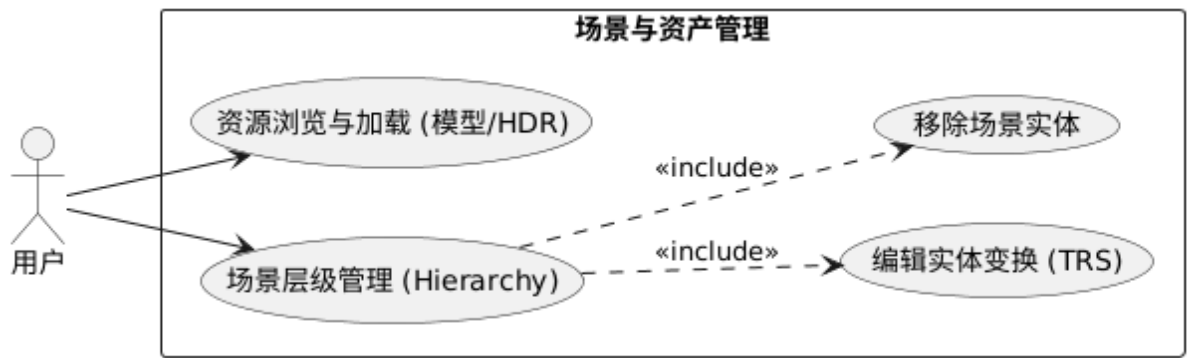


图 3.1 场景与资产管理用例图

（1）UI 界面

用户可以对渲染管线和环境光照进行深度定制，以达到最佳的画质表现：

- 渲染路径与视图切换：用户可以在光栅化（Forward）、混合渲染（Hybrid）及路径追踪模式间实时切换。提供多种可视化模式，允许用户查看 Albedo、法线、材质参数、运动矢量以及 SVGF 方差图等中间附件。
- 渲染调试控制：提供对光线追踪阴影、反射等独立控制的开关。针对信号重建，支持独立启禁用 SVGF 时空降噪与 TAA 抗锯齿，以便直观对比画质改善效果。
- 光源控制：支持对主光源的方向、颜色进行调节，加载环境光照。
- Debug 信息查看：提供相机参数，帧数统计，性能分析等信息的实时显示，显示每个 Pass 的耗时，以及 RenderGraph 的 DAG 拓扑结构导出。

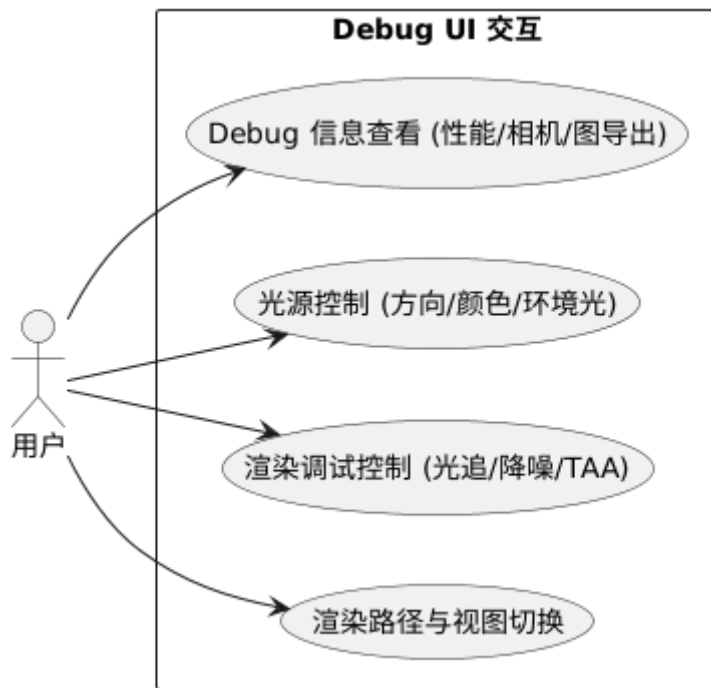


图 3.2 渲染控制与参数配置用例图

（2）场景交互与观察模块

该模块为用户提供灵活的场景巡检手段与实时的系统状态反馈：

- 复合相机交互：系统实现了一套基于弧球（Arcball）逻辑的编辑器相机。用户可通过“键盘 + 鼠标”的组合键模式进行操作：按住 Alt 键配合鼠标左键实现视角的环绕旋转，配合中键实现焦点的平移，配合右键或滚轮实现焦距缩放。该模式确保了用户能以任意角度观察场景细节。
- 交互状态反馈：Debug UI 面板实时同步并显示相机的物理参数，包括相机的世界坐标（Position）、观察焦点（Focal Point）以及视场角（FOV），方便用户在调试特定渲染效果时精确记录机位。
- 系统性能监测：集成实时的性能分析器，显示当前帧耗时（Frame Time）与帧率（FPS）波动。通过内置的时间戳查询技术，系统能够将渲染管线的内部执行细节可视化，允许用户监控每个渲染 Pass 的 GPU 执行开销及占比。

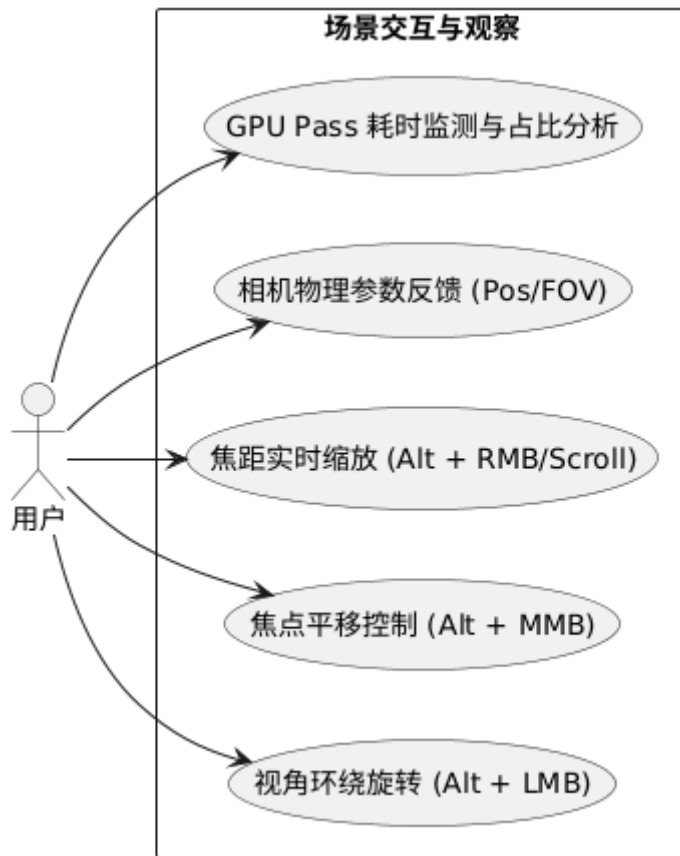


图 3.3 场景交互与观察用例图

3.4 系统非功能性需求分析

非功能性需求定义了系统在执行功能时所必须遵循的质量约束与技术指标。针对实时混合渲染系统的特性，本项目从性能、安全性、可用性、易用性以及可维护扩展性五个维度确立了非功能需求基准。

3.4.1 性能和响应时间：

实时渲染系统对计算效率有着严苛的要求，需确保算法逻辑与数据流转的高度优化：

- 交互实时性指标：在目标硬件平台上，系统应能维持稳定的亚秒级反馈。单帧渲染的端到端延迟应满足实时渲染的工业标准，确保在复杂场景漫游与参数调节时具有平滑的交互响应。
- 算法复杂度控制：系统需针对计算密集型任务（如时空信号降噪）进行时间与空间复杂度的深度优化。利用并行计算等技术规避性能瓶颈，确保系统性能不随场景复杂度的增长而出现非线性骤降。

3.4.2 安全性与稳定性

在底层图形 API 驱动下，渲染系统必须构建坚实的资源保护与执行同步机制：

- 底层资源安全：系统需实现一套基于引用计数与延迟销毁的显存管理机制。通过 RenderGraph 的资源生存区间分析，确保 GPU 正在引用的缓冲区或图像资源不会被非法释放，从架构层面预防驱动层面的访问冲突。

- 执行同步健壮性：系统需要规范管理异步流水线中的资源依赖。通过自动化同步引擎推导并生成最优的同步屏障，避免由于读写竞态导致的画面闪烁或硬件超时重置。

3.4.3 可用性与可靠性

系统应具备在复杂运行环境下保持稳定输出的能力，并提供有效的容错逻辑：

- 状态切换可靠性：在动态切换渲染路径（如从光栅化切至光线追踪）或调整配置参数时，系统需具备健壮的状态机管理能力，确保管线重建过程中的显存状态转换合法，不发生崩溃。

- 异常监控与诊断：系统需要有良好的监测系统，能够实时捕获硬件特性缺失（如不支持 RT 扩展）或着色器编译异常，并向开发者提供详细的底层诊断信息。

3.4.4 易用性

系统界面设计与交互逻辑应符合图形行业规范，降低操作者的认知负担：

- 交互直觉性：参数调节面板与场景管理接口的设计应遵循数字内容创作工具的通用逻辑。常用渲染配置应提供直观的可视化调节手段，实现“所见即所得”的操作体验。

- 辅助支持能力：系统需配备完善的实时日志监测与诊断工具，并提供符合行业标准的摄像机控制系统，确保用户能够低成本、高效率地进行场景观察与算法验证。

3.4.5 可维护性与可扩展性

渲染系统的架构设计应支持长期的功能迭代与模块替换：

- 高度模块化解耦：系统需采用逻辑层与驱动层分离的架构。通过高度抽象的调度中枢，使得新的渲染算法节点能够以插件化的方式接入，而无需对核心同步框架进行破坏性修改。

- 架构透明度：系统需维持清晰的代码结构，确保其内部的数据流向与组件协作关系易于理解，从而降低二次开发与性能调优过程中的维护成本。

3.5 系统核心类设计

本节通过一系列类图展示系统的静态设计结构，涵盖从全局框架到具体渲染调度的协作关系。系统采用高度模块化的 C++ 设计，其核心类协作主要分布在框架管理、资源调度、场景组织及渲染执行四个主要维度。

3.5.1 全局管理与框架类设计

系统以 `Application` 为单例核心，通过组合模式聚合 `Window` 和 `VulkanContext` 确立运行环境。分层框架利用 `std::vector<std::shared_ptr<Layer>>` 构建逻辑栈，支持 `EditorLayer` 等功能层级的一致性分发。其中 `VulkanContext` 对 `VulkanInstance` 与 `VulkanDevice` 进行了深度封装，为全系统提供统一的硬件能力抽象。

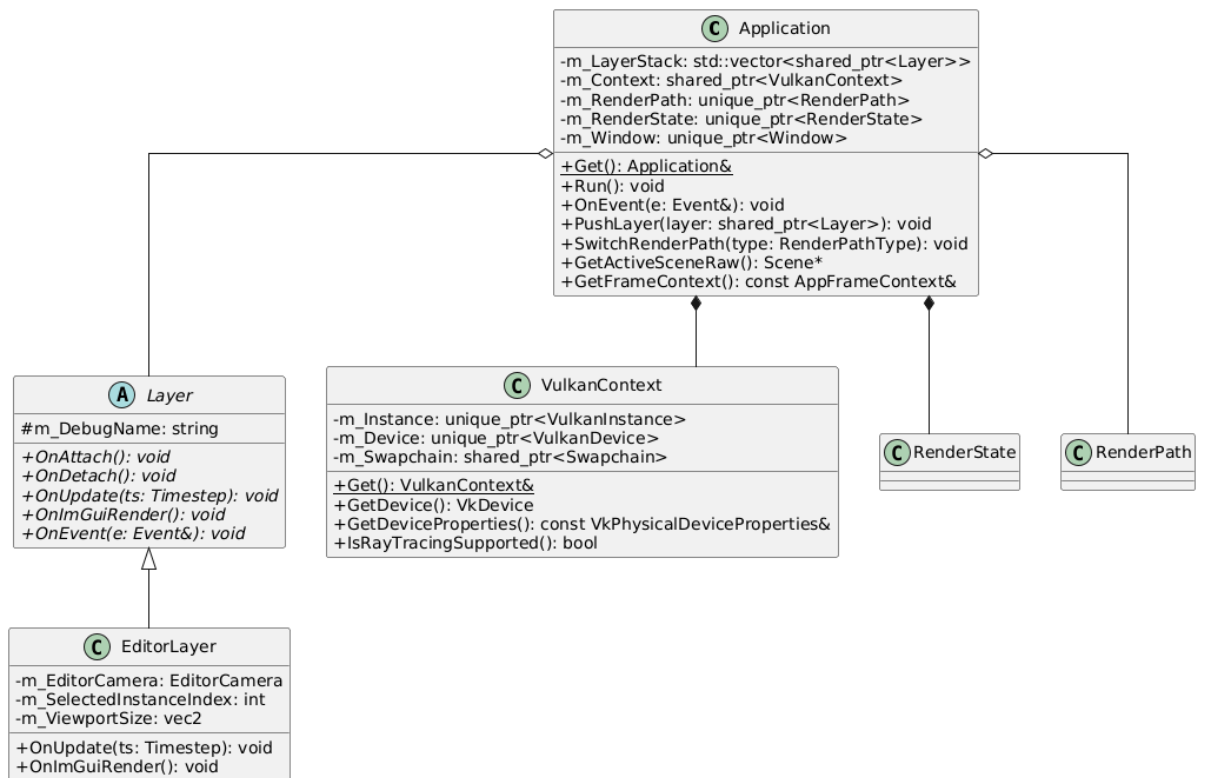


图 3.4 全局管理与分层框架类图

3.5.2 资源管理与场景组织类设计

系统采用组件化(Component-based)的场景组织结构。`ResourceManager` 负责 Vulkan 资源的 RAII 封装，通过基于线程池(Thread Pool)的多线程后台处理流水线，实现资产加载与逻辑更新的并行化，确保了大规模场景导入时的系统响应效率。

Scene 类通过 Entity 容器组织场景对象，每个实体由 TransformComponent（TRS 变换）与 MeshComponent（模型引用）构成。系统通过对这些组件的实时追踪，自动维护并同步硬件加速结构（AS）。

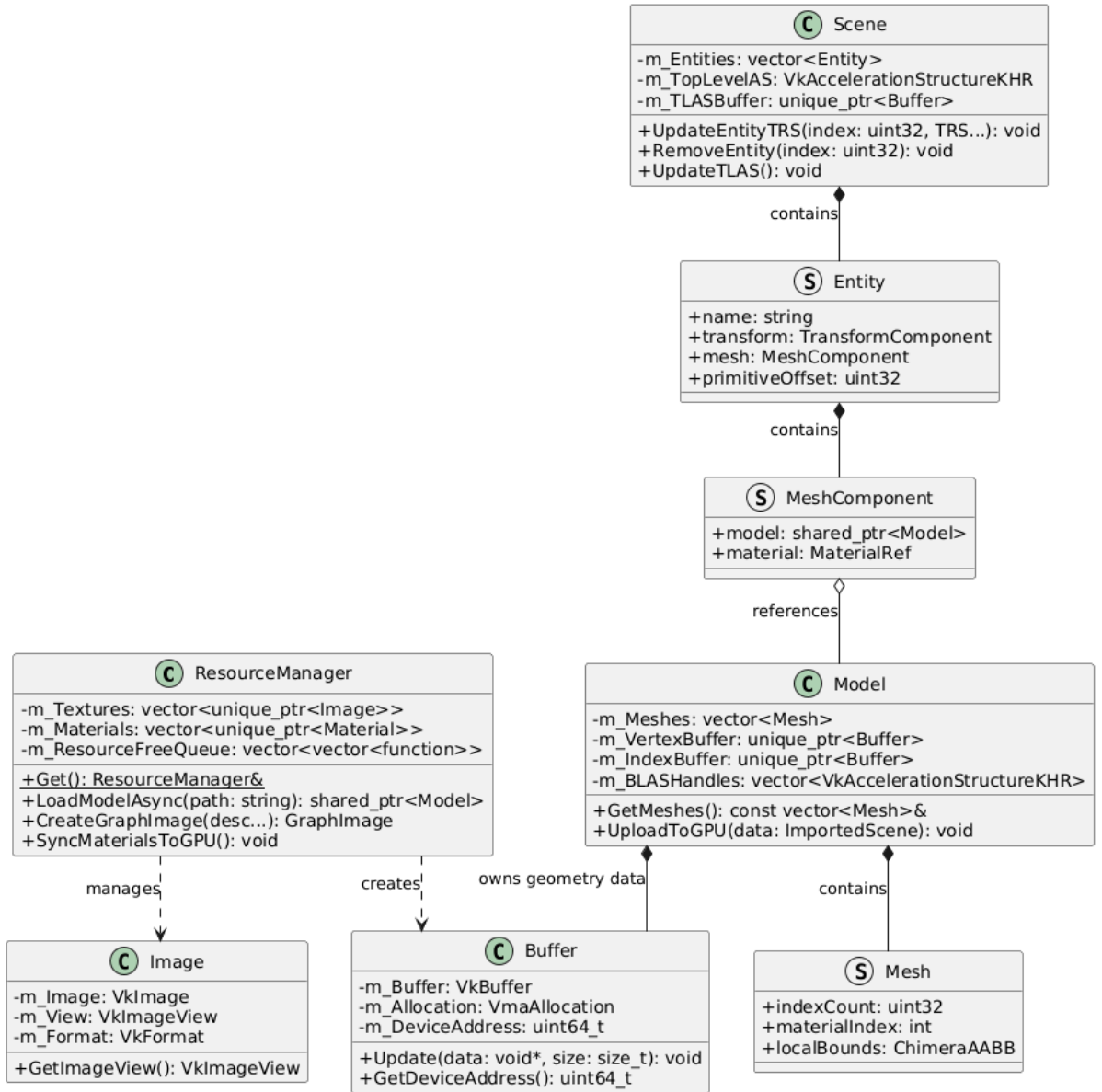


图 3.5 资源管理与场景组织类图

3.5.3 渲染调度引擎核心类设计

渲染调度由声明式的 **RenderGraph** 引擎驱动。系统通过 **PassBuilder** 捕获各渲染通道对显存资源的读写（Read/Write）依赖，从而推导出资源的物理生存区间。在执行阶段，引擎为不同类型的通道分发对应的执行上下文（**ExecutionContext**），屏蔽了底层同

步屏障与镜像布局转换的复杂性，实现了高层次的架构解耦。

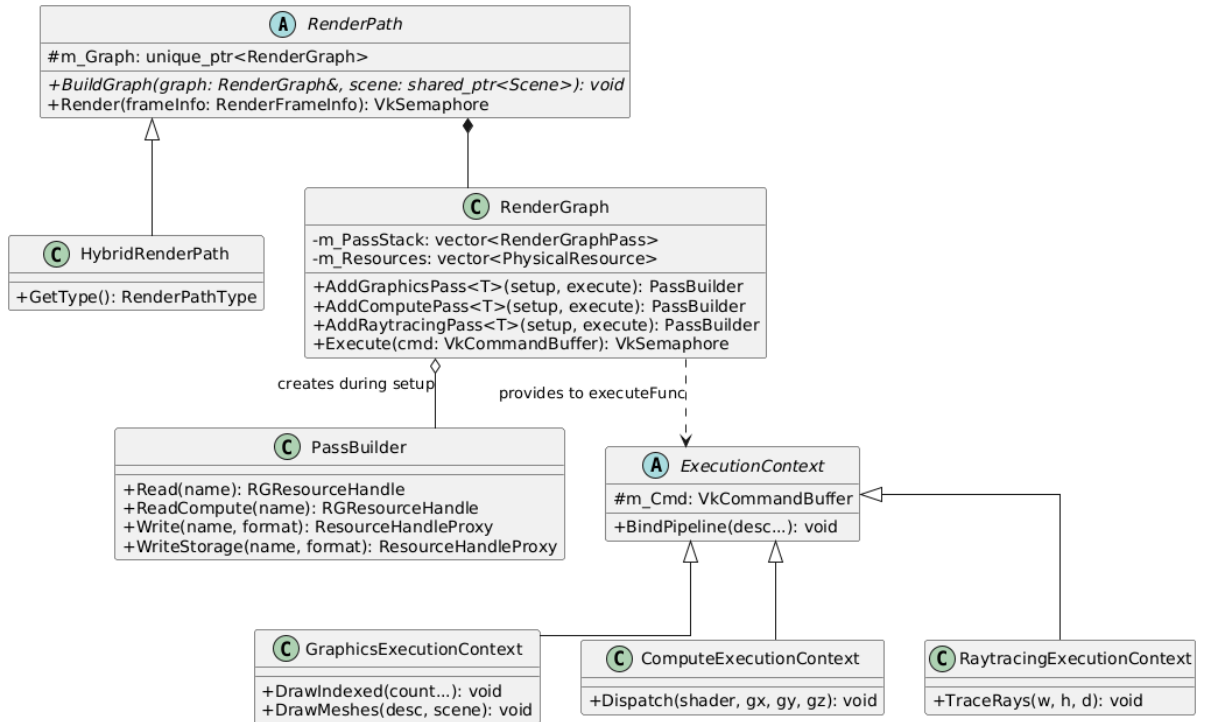


图 3.6 RenderGraph 调度引擎类图

3.6 本章小结

本章通过对可行性、业务流程及功能模块的深度拆解，首先，通过多维度的可行性论证，确认了本课题在当前开源生态与硬件技术基础上的实施合理性。其次，详细梳理了系统的核心业务流转过程，并通过抽象用例确立了引擎在场景管理与交互调优方面的用户功能基准。并通过详细的类图设计勾勒出了系统各子模块间的协作逻辑。本章的研究成果为下一章进入概要架构设计提供了明确的功能指导与工程依据。

第四章 概要设计

本章将从软件工程的视角出发，对混合渲染引擎进行概要设计。从参考引擎和优秀设计出发，借鉴并设计出本引擎的架构与模块。

4.1 引擎的架构设计

随着多年的发展，目前有许多成熟的引擎可供参考。本项目在底层架构上参考了结构清晰的 Walnut 框架，而在核心图形调度上则引入了现代化的 RenderGraph 系统。本节将对这两种核心技术架构进行理论分析，并引出其在本项目中的设计。

4.1.1 Walnut

Walnut 是从游戏引擎 Hazel 中抽象出来的微内核框架，它剔除了游戏引擎中繁重的物理、音频、脚本等模块，仅保留了最核心的渲染循环与窗口管理逻辑。

Walnut 使用了微内核与层叠式架构（Layer Stack）。如图 4.1 所示，整个框架被严格划分为应用外壳、逻辑层栈和平台抽象层。其关键机制在于确立了基于生命周期回调函数（Lifecycle Callbacks，如重载 OnUpdate、OnRender、OnImGuiRender 方法）的渲染管线切入与事件分发系统。框架在每一帧通过自底向上的遍历，依次触发各逻辑层的回调，实现了渲染画面与交互调试 UI 的分离，也让本项目在开发混合渲染管线时，无需关注底层的窗口重建与系统级事件路由细节。

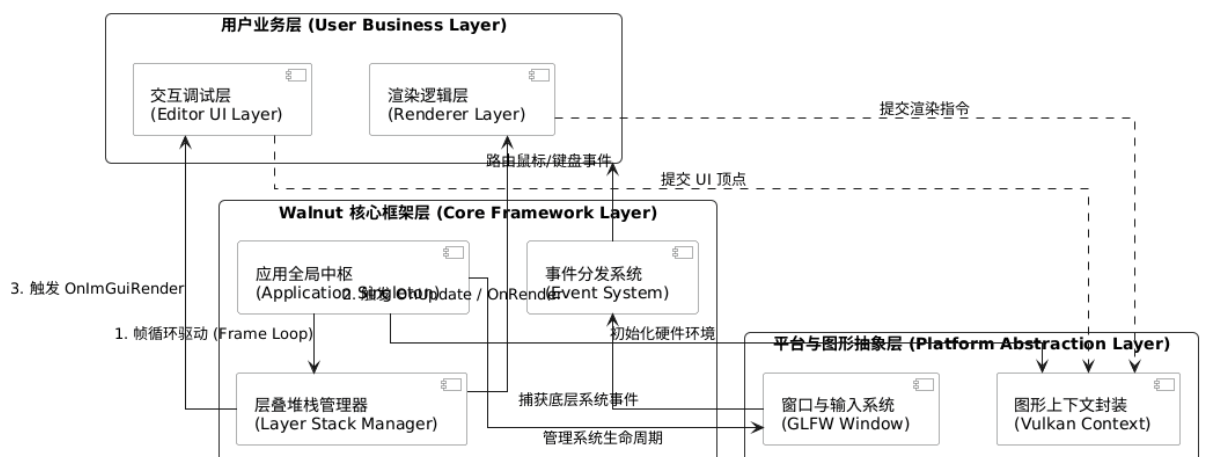


图 4.1 Walnut 框架组件架构图

4.1.2 RenderGraph

现代图形 API 将内存管理、多线程管理等大部分工作交由开发者负责，这赋予了开发者极高的硬件控制自由度，但也呈指数级增加了 API 使用的复杂度和困难度。例如，仅利用原生 Vulkan 绘制单一基础图元便需构建上千行的前置代码。

软件工程的核心诉求之一即为复杂度管理。现代引擎必须实现一个中间层架构，在利用图形 API 性能优势的同时，有效降低其复杂度，并让整体渲染管线更加可扩展、易于调试。此类作为渲染调度中枢的架构即为渲染图（RenderGraph）。

RenderGraph（也叫 Frame Graph）最早由 EA 寒霜引擎（Frostbite Engine）团队在 GDC 2017 大会上首次提出。它主要由 RenderPass 和 Resource 构成：

RenderPass 是一份元数据，定义了一个完整的渲染流程需要用到的所有资源。

Resource 是指 RenderPass 使用的 RenderTarget、Texture、Shader 等物理资产。

每个 RenderPass 都会显式指定对应的 Input 和 Output 资源。通过这种依赖关系，RenderPass 和 Resource 就形成了一个严密的有向无环图（DAG）结构。

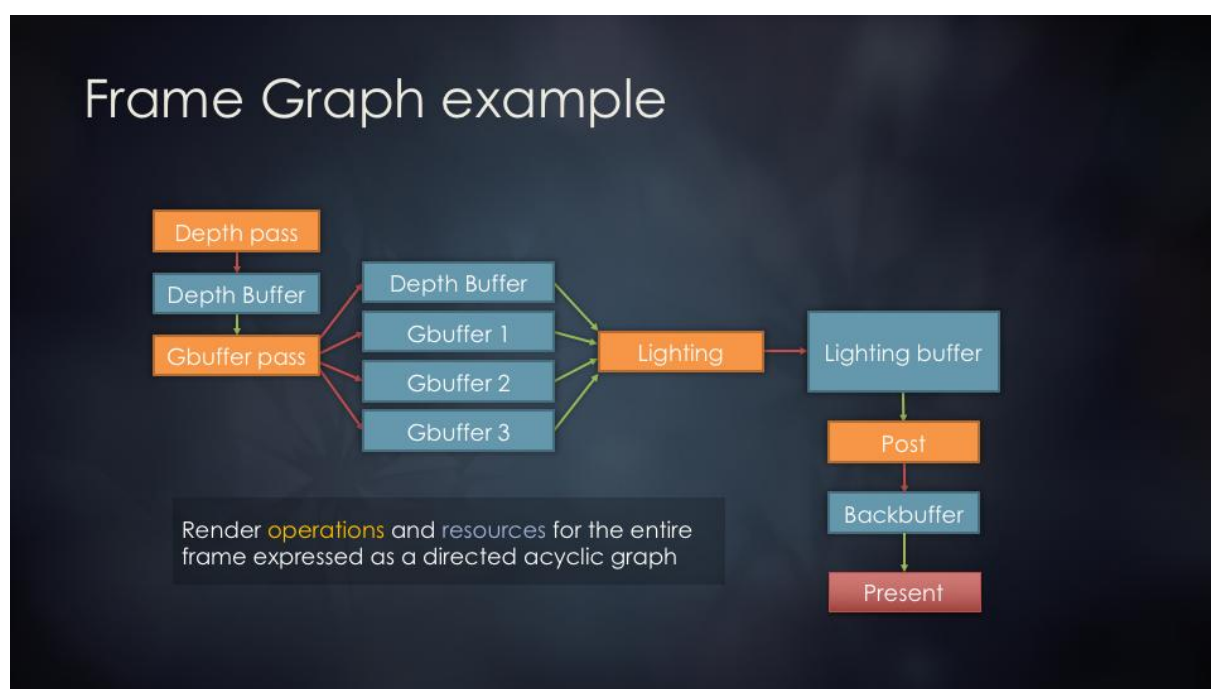


图 4.2 RenderGraph DAG 可视化示例图

Render Graph 拥有完整一帧的全部信息，通过分析各个 RenderPass 之间依赖关系可以方便得对资源做优化。DAG 也可以很方便的做到可视化，方便对现代复杂渲染管线的调试。可以自动化推导 Pass 执行顺序、自动插入同步屏障，并利用显存复用

（Memory Aliasing）减少整体的内存占用。

4.1.3 本引擎架构设计

结合上述对 Walnut 架构与 RenderGraph 调度架构的理论分析，本项目使用了下面图 4.3 中的五层拓扑架构体系

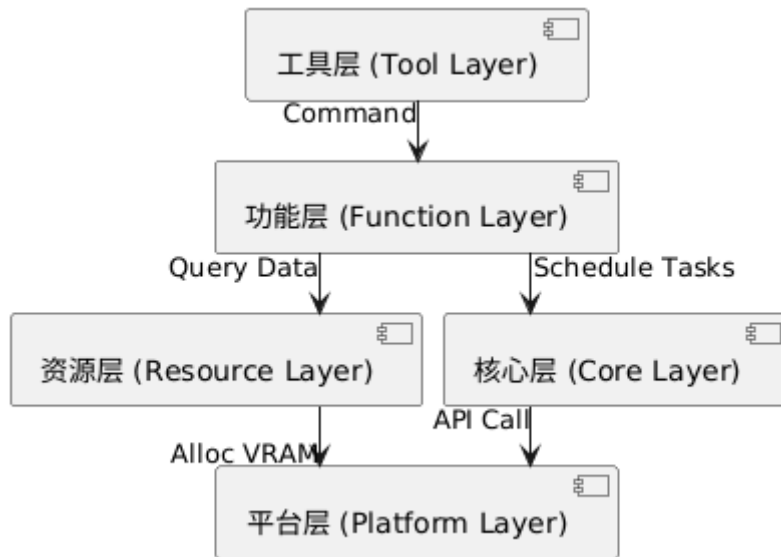


图 4.3 引擎五层分层拓扑架构图

1. 平台层 (Platform Layer): 位于架构最底层，引擎的硬件抽象层 (HAL)。其核心职责是定义系统与底层图形 API 及操作系统之间的通信边界，该层参考了 Walnut 框架的平台抽象思想，确保上层逻辑不受特定显卡硬件或系统驱动细节的影响。

2. 核心层 (Core Layer): 提供全局基础设施服务。该层专注于为全引擎提供底层的任务并行调度、显存/内存原始分配策略以及基础数学运算支持，是整个系统运行的基石。

3. 资源层 (Resource Layer): 负责资产的生命周期管理与数据组织。其职责是建立一套标准的数据加载与存储规范，将磁盘上的原始资源（模型、贴图等）转化为引擎可高效访问的内存对象，并负责动态维护光线追踪所需的顶层/底层硬件加速结构 (TLAS/BLAS)。

4. 功能层 (Function Layer): 引擎的渲染算法驱动核心。该层承载了 RenderGraph 调度系统，负责协调资源层的数据与核心层的计算能力，实现具体的混合渲染管线调度、驱动渲染管线，将抽象的渲染图元转化为真实的物理光影。

5. 工具层 (Tool Layer): 位于架构顶端，直接面向用户。其通过调用功能层提供的

接口，实现可视化的资源加载，交互调试、性能监控以及渲染参数设置等功能。

4.2 功能模块划分

在五层垂直架构的基础上，本节将系统按业务逻辑划分为六个核心功能模块。这些模块通过跨层协作，共同完成从底层资源托管、场景调度到上层光影合成与性能剖析的复杂任务。

4.2.1 图形硬件后端模块

该模块主要承担抹平图形 API 复杂度、提供显式硬件控制的核心职责。其功能涵盖三个维度：一是图形上下文建立，负责逻辑设备初始化及呈现队列(Swapchain)管理；二是显存池化管理，利用核心层设施实现基于 VMA (Vulkan Memory Allocator) 的高效显存亚分配，规避驱动层频繁申请导致的性能波动；三是描述符布局自动推导，为后续光追管线提供底层的描述符资源池 (Descriptor Pool) 支持。

4.2.2 资源管理模块

该模块专职负责海量三维资产的摄入与显存流转。其核心功能是构建非阻塞式的资产加载流水线，将 glTF 2.0 等标准资产解析为硬件可极速访问的顶点缓冲与图像对象。针对混合渲染需要随机访问全局材质的严苛特性，该模块实现了无绑定 (Bindless) 材质资源管理系统，将场景中所有的纹理资源统一映射至全局描述符数组中，从底层打破了传统渲染器逐对象绑定 (Per-Object Binding) 的性能瓶颈。

4.2.3 场景管理模块

该模块负责维护物理世界的空间拓扑一致性，是驱动硬件光线追踪的绝对先决条件。其核心功能分为两部分：一是空间划分与视锥体剔除，利用包围盒 (AABB) 算法筛选出当前视角的活跃实体，减少冗余绘制开销；二是加速结构动态维护，负责实时追踪场景内实体的运动位移，并动态重构底层几何结构 (BLAS) 与顶层场景结构 (TLAS)，确保 GPU 的光线探测单元 (RT Core) 能够时刻感知绝对精确的物理世界分布。

4.2.4 渲染调度模块

该模块是引擎实现“数据驱动渲染”的思想大脑。它完全围绕 RenderGraph (渲染任务图) 展开：通过在 CPU 端收集各项渲染意图并构建有向无环图 (DAG)，在 GPU 提交前执行拓扑排序，自动化推导出最优的通道 (Pass) 执行序列。其核心价值在于消除了手动维护 Vulkan 图像内存屏障 (Image Memory Barriers) 的沉重工程负担，并实

现了生命周期不重叠资源的智能显存复用（Aliasing）。

4.2.5 混合渲染管线模块

该模块是引擎图形算法的集中体现。它将混合渲染流程解构为四个高度解耦的核心子单元：几何特征采集单元（生成 G-Buffer）、光影探测单元（执行 1-SPP 硬件光追采样）、时空信号重建单元（执行 SVGF 时空降噪）以及光照合成单元（基于 PBR 的物理着色）。每个单元均以独立节点的身份挂载于渲染调度模块中，共同完成从几何到光影的转换。

4.2.6 交互调试模块

该模块位于架构顶端，旨在为渲染算法开发者提供一个非侵入式、高保真的辅助分析环境。其功能设计划分为两大核心维度：一是“算法实时反控”，允许用户在前端控制面板动态调整核心参数（如光追采样数、SVGF 降噪时空权重、曝光度等），并即刻反馈于最终画面，实现参数的闭环调优；二是“管线性能侦测”，通过在 RenderGraph 的关键节点埋点插入 Vulkan 时间戳查询(Timestamp Queries)，实时捕捉并可视化输出 GPU 计算节点的微秒级耗时明细，为混合管线的算力瓶颈定位提供最权威的数据支撑

4.3 混合渲染管线流程设计

在明确了引擎的模块化分工后，本节将深入探讨由渲染调度模块驱动的核心业务逻辑——混合渲染管线（Hybrid Rendering Pipeline）。该管线的设计目标是打破传统前向渲染（Forward Rendering）算力浪费的局限，构建一条兼顾执行效率与物理真实感的数据流转通道。

整个混合管线在宏观架构上被严格解耦为四个依次递进的概念阶段（如图 4.4 所示），前一阶段的输出将作为后一阶段的输入先验条件。

。

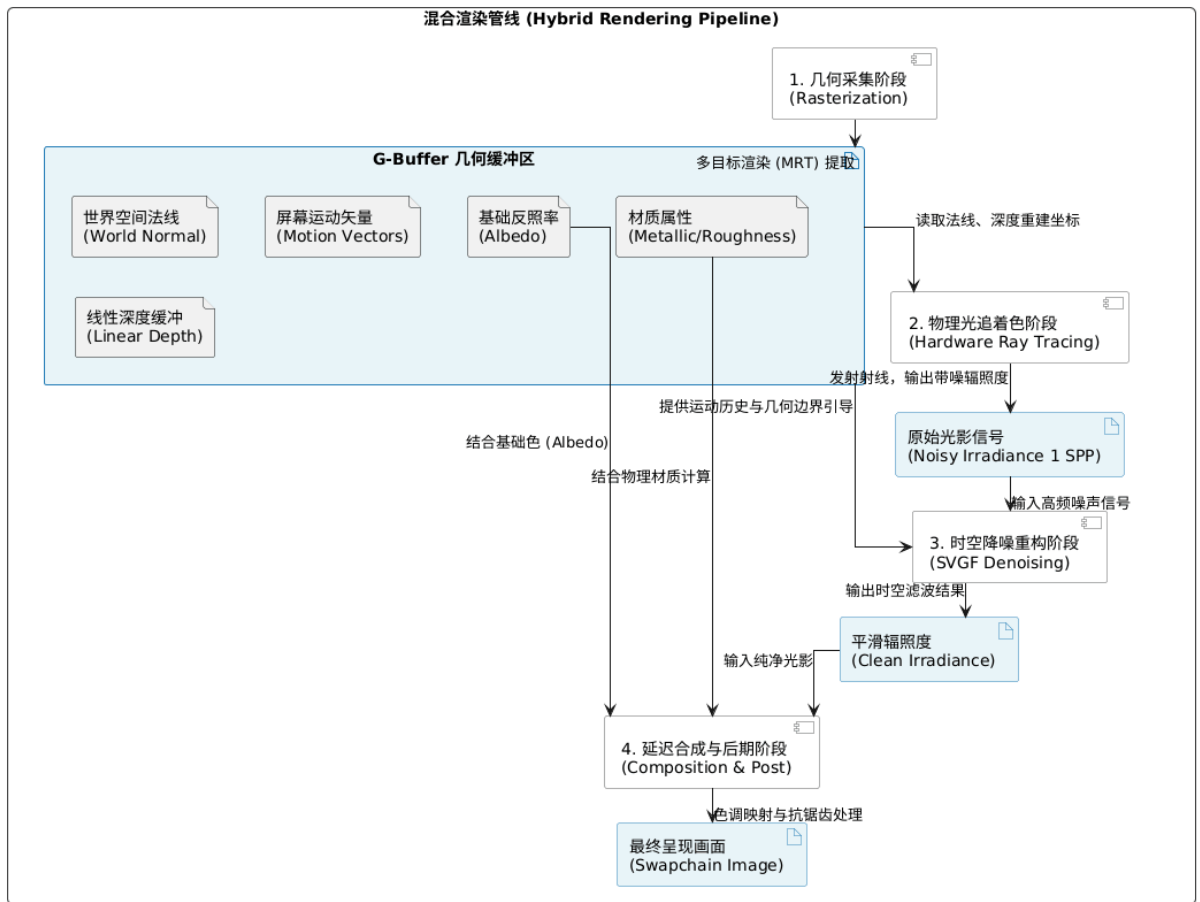


图 4.4 混合渲染管线数据流转图

4.3.1 几何特征采集

混合管线的首个环节是建立对当前视角的“屏幕空间几何认知”。系统在此阶段利用光栅化技术，不进行复杂的光照解算，专职剥离场景的几何体信息与材质属性。通过将深度、法线、反照率 (Albedo) 及运动轨迹 (Motion Vectors) 分离存储至几何缓冲 (G-Buffer) 中，管线成功将“几何处理”与“光照计算”彻底解耦。

4.3.2 物理光影计算

在获取了屏幕空间的几何信息后，管线进入光影解算核心。该阶段利用采集到的表面特征，在三维空间中向光源及周围环境发射探测射线，模拟光线的遮蔽、反射与衰减。由于采用了 1-SPP 等低采样率策略以换取实时性，本阶段输出的原始光影信号伴有随机噪声。

4.3.3 时空信号重建

为了消除低采样率探测带来的画面瑕疵，管线引入了信号重建阶段。该阶段集成

SVGF（时空方差引导滤波）算法，复用第一阶段提取的运动轨迹进行历史数据累积，并结合当前帧的几何特征执行边缘保留的空间滤波。其目的在于以较低的开销还原出等效于高采样率下的纯净光照结果。

4.3.4 光照合成与后期处理

在获取纯净的光影分量后，管线进入最终闭环。合成模块基于物理渲染模型（PBR），将基础颜色与重构后的物理光影进行合并。最后，画面将流经后期处理管线，完成时域抗锯齿（TAA）稳定与色调映射（Tone Mapping），输出供显示设备呈现的最终二维图像。

4.4 本章小结

本章完成了引擎的宏观架构设计。通过五层分层拓扑架构确立了系统的解耦原则，并详细定义了图形后端、资产管理、场景加速及渲染调度等核心功能模块的逻辑职责。为后续章节深入探讨核心组件的底层运行机制与算法代码落地提供了坚实的工程基础。

第五章 详细设计

在第四章的概要设计中，本文已经确立了混合渲染引擎的总体架构与宏观模块划分。第五章作为本课题的核心环节，将立足于详细设计与具体工程落地，深入探讨各子系统的底层运行机制与算法实现。

5.1 图形硬件后端模块

图形硬件后端是整个混合渲染引擎的物理基座。由于现代显式图形 API（如 Vulkan、DirectX 12）采用了极度轻量化的驱动设计哲学，将内存管理、同步原语与多线程状态机的控制权全面移交给了应用层。为了避免底层繁琐的状态机操作反向污染上层的渲染业务逻辑，本模块通过高度封装的组件化设计模式，构建了稳定、高效且线程安全的底层图形上下文。

5.1.1 基于 volk 的动态函数指针加载机制

在静态结构设计上，本模块采用了外观模式（Facade Pattern）与系统组件化思想。系统并未将所有功能塞入一个臃肿的“上帝类”，而是设计了 `VulkanContext` 作为全局唯一的外观枢纽。该枢纽将底层复杂的物理设备、显存分配器、交换链及多线程指令池封装为互相协作的子组件，对上层屏蔽了所有 Vulkan 原生句柄的获取与管理逻辑。

以下为图形硬件后端模块的静态组件拓扑图：

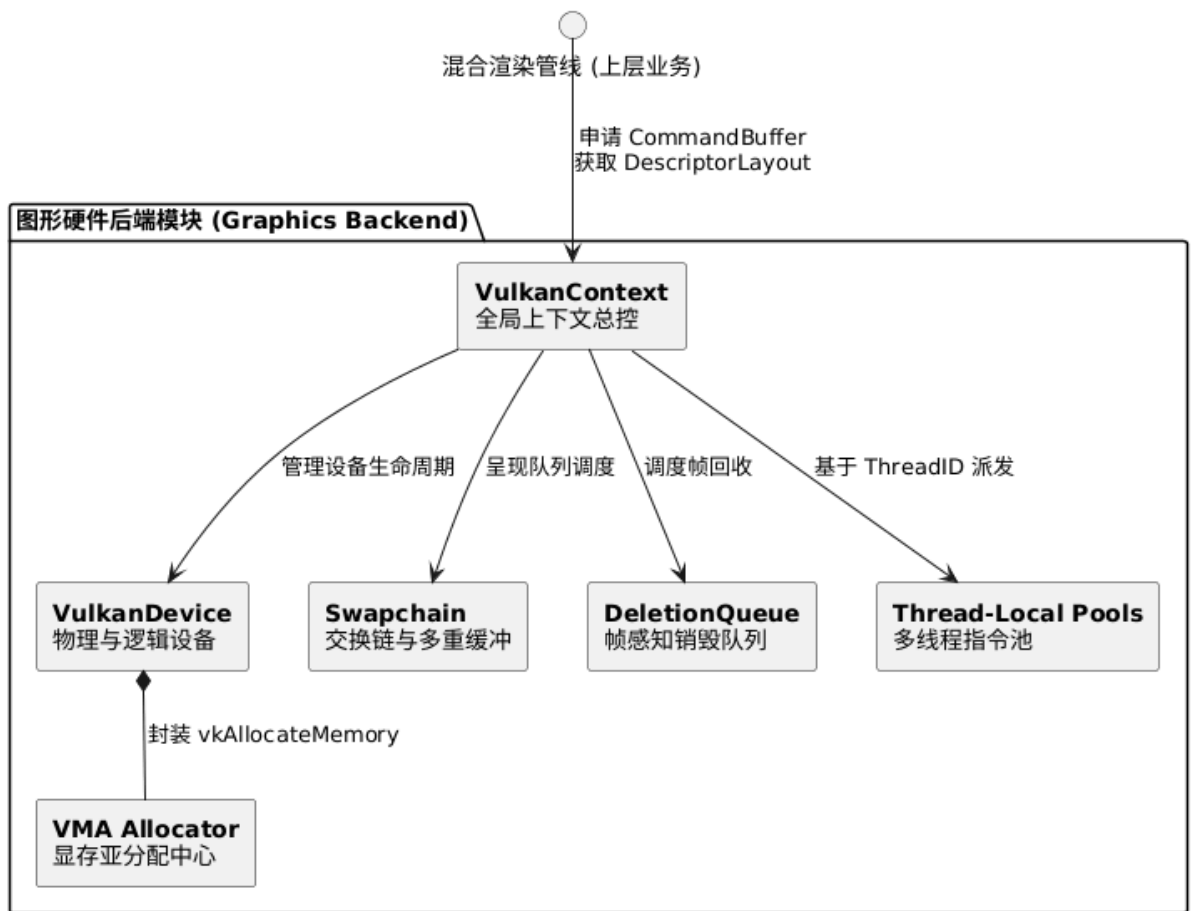


图 5.1 图形硬件后端模块静态组件拓扑图

现代渲染引擎必须充分利用多核 CPU 的算力进行渲染指令的并行录制。然而，Vulkan 的指令池（CommandPool）在底层并不具备线程安全性。为此，系统摒弃了单指令池加锁的低效方案，在 VulkanContext 中维护了一个基于当前线程 ID（ThreadID）的动态派发组件。它为每一个异步工作线程（如主渲染线程、资源异步加载线程）动态分配独立的 CommandPool，从底层彻底消除了多线程并发录制渲染指令时的锁竞争（Lock Contention）瓶颈。

原生 Vulkan 的 vkAllocateMemory 对最大分配次数有严格的硬件上限（通常为 4096 次）。VulkanDevice 组件内部深度集成了业界标准的 AMD VMA（Vulkan Memory Allocator）。上层的任何 Buffer 或 Image 创建请求，均会被路由至该组件，由 VMA 进行大块物理显存的亚分配（Sub-allocation），有效遏制了高频资源创建带来的显存碎片化。

5.1.2 物理特性按需注入机制

混合渲染管线对 GPU 的硬件能力提出了极高的要求（如 1-SPP 硬件光线查询、无绑定描述符等）。在逻辑设备的初始化过程中，本模块利用 Vulkan 的 pNext 指针链表机制，设计了动态特性扩展注入时序。

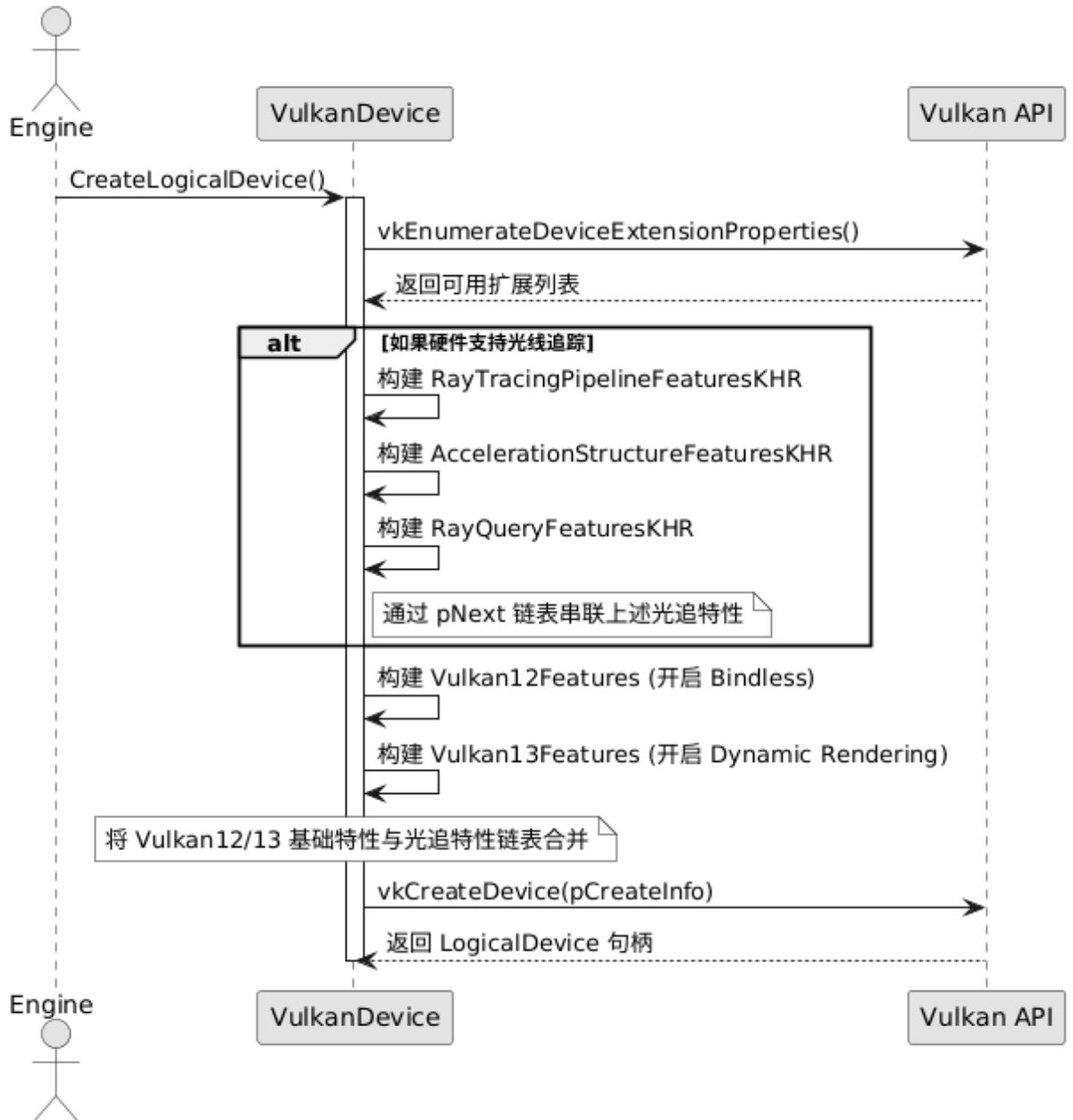


图 5.2 逻辑设备光线追踪特性动态注入时序图

该时序设计保证了系统的极高鲁棒性。引擎在启动时会通过硬件探测评估物理设备的算力。若设备支持硬件光追扩展（如 RTX 系列显卡），系统会通过 `pNext` 结构体指

针链将光追管线（Ray Tracing Pipeline）与光线查询（Ray Query）能力动态注入设备上上下文；若设备不支持，则安全降级，保障了基础渲染管线的正常初始化，避免了硬编码导致的环境崩溃。

5.1.3 交换链（Swapchain）管理与异常恢复

在现代三重缓冲（Triple-Buffering, MaxFramesInFlight=3）的渲染架构中，CPU 的指令提交进度通常领先于 GPU 实际渲染进度 1 到 2 帧。若 CPU 在当前帧直接释放被剔除实体或卸载场景的显存，而此时 GPU 仍在上一帧的指令缓冲中读取该资源，将直接触发 Device Lost 级硬件宕机。

针对此时间轴错位问题，本模块设计了帧感知延迟销毁队列（Frame-Aware Deletion Queue）机制。

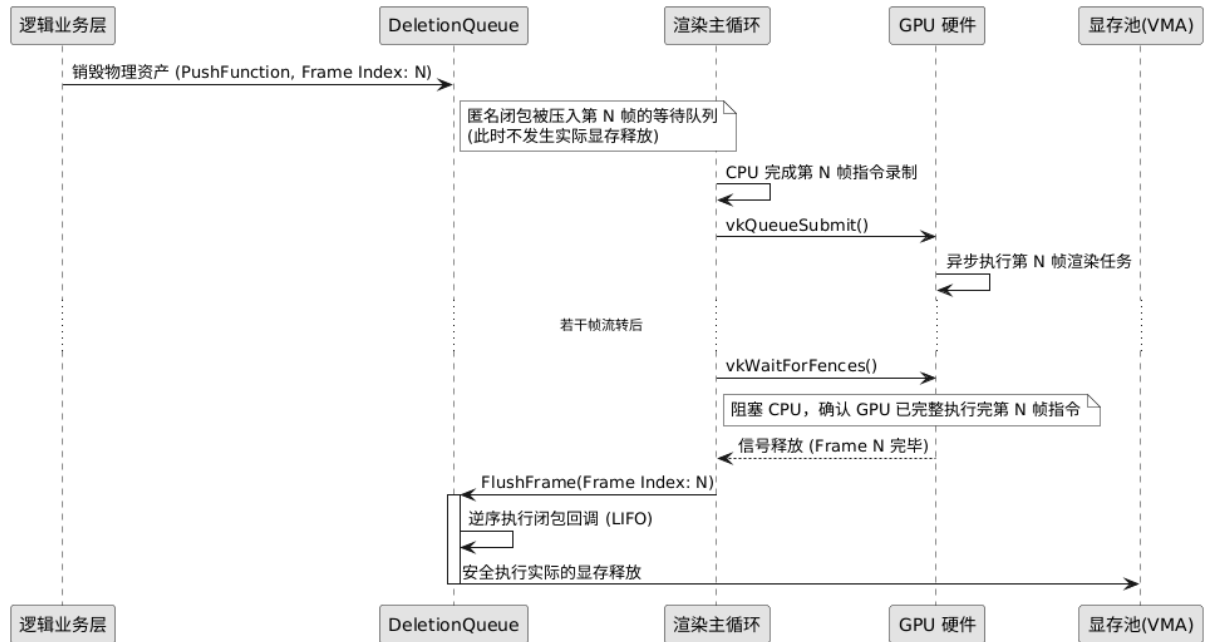


图 5.3 帧感知显存延迟回收机制时序图

5.2 资源管理模块

在混合渲染管线中，海量三维资产（如高精度 glTF 模型、PBR 贴图阵列）的加载与显存流转是影响系统整体性能的关键环节。特别是在硬件光线追踪阶段，光线求交测试要求管线能够在一次调度中随机访问全场景材质。传统的“逐对象绑定（Per-Object Binding）”模式难以满足单次渲染流中高频次、无规律的材质寻址需求。

针对这一要求，本模块引入了无绑定（Bindless）架构设计与基于 RAII 的异步引用

计数机制，旨在构建一套高效且安全的资源调度系统。

5.2.1 静态架构：无绑定(Bindless)资源池设计

为了满足光线追踪阶段全局材质的随机访问需求，本系统在 `ResourceManager` 中改变了为每个模型独立分配描述符集（Descriptor Set）的传统方案。系统在初始化阶段统一向图形硬件后端申请了全局描述符数组，其中包含独立的存储缓冲（SSBO）数组与采样器图像（`sampler2D`）数组。

以下为无绑定资源管理架构的静态组件设计图：

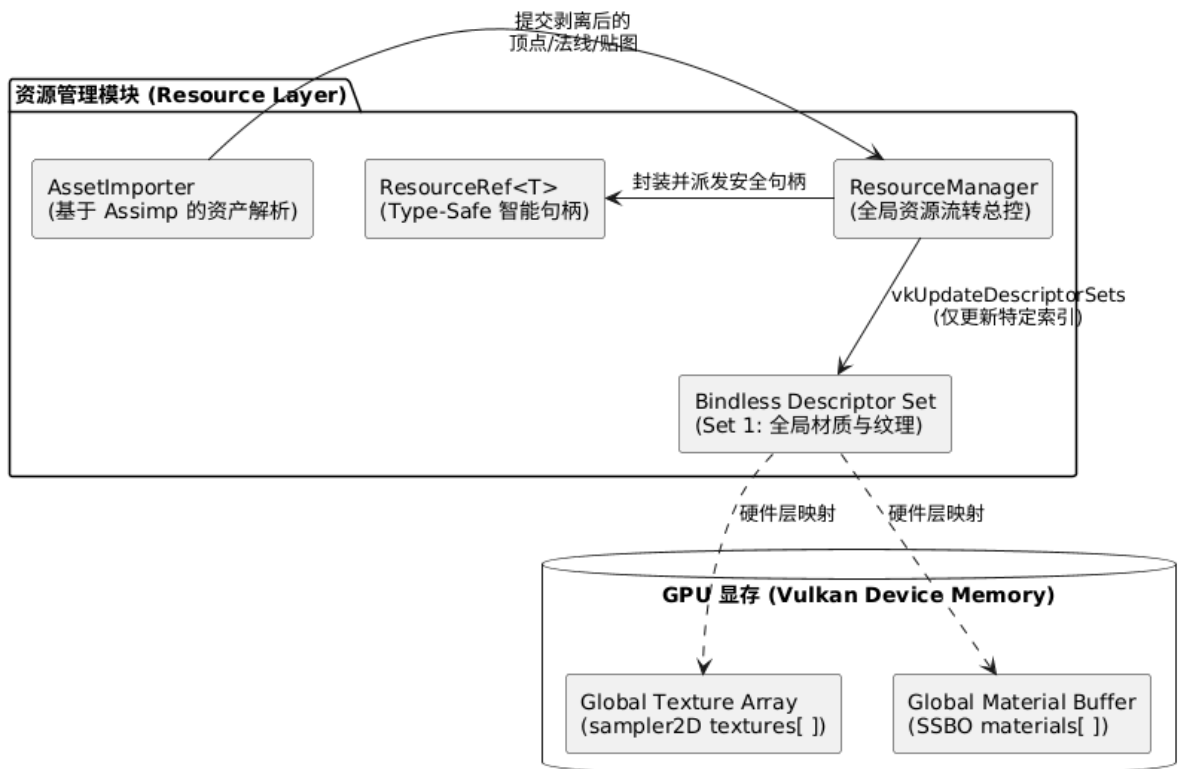


图 5.4 资源管理无绑定(Bindless)组件拓扑图

在该架构下，任意一个 `Material` 对象不再持有具体的 Vulkan 图像视图原生句柄，而是持有一个无符号整型索引（`TextureHandle.id`）。在着色器（Shader）中，光线命中表面后，通过读取该索引，即可直接从全局数组中提取相应的 `Albedo` 或 `Normal` 数据。此举将 CPU 端的绑定开销（Binding Overhead）彻底清零，为混合管线扫清了底层障碍。

5.2.2 动态加载：基于引用计数的异步流转机制

为保证在动态加载大型外部资产时主渲染循环的流畅性，本系统在 `AssetImporter` 中实现了基于 `std::future` 的多线程异步加载机制。

异步加载的主要技术难点在于显存生命周期的并发管理。传统的显式资源释放机制在场景动态切换或实体剔除时，容易引发内存泄漏或悬空指针问题。针对此问题，本模块设计并实现了基于引用计数与延迟回收的 `ResourceRef<T>` 模板类。

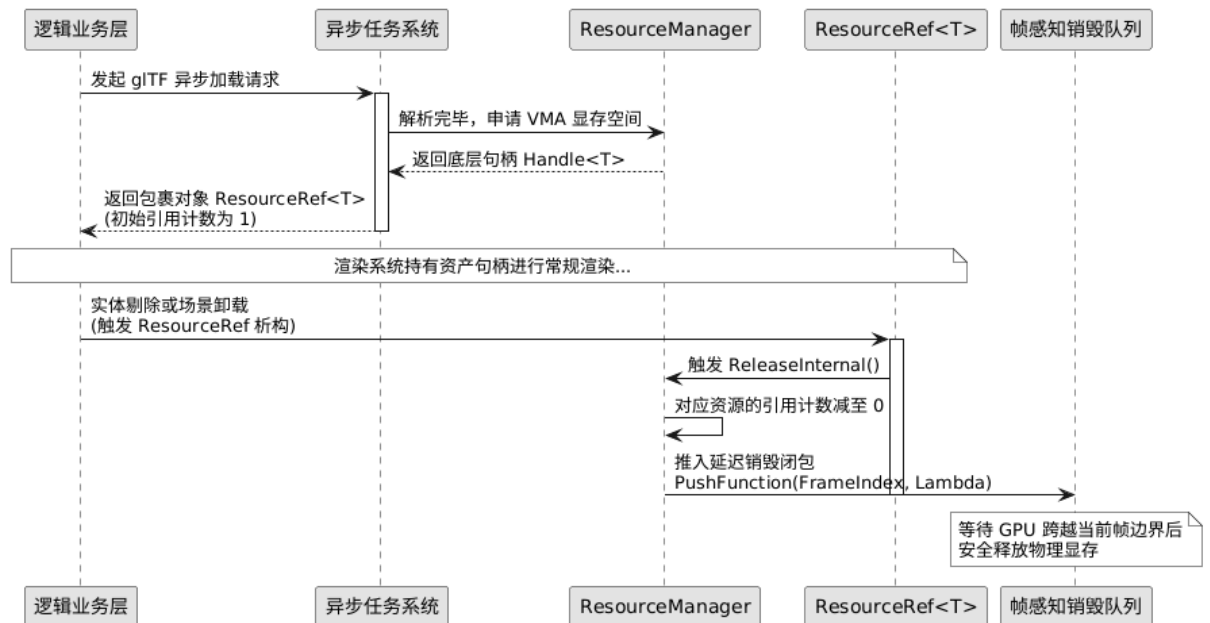


图 5.5 基于引用计数的异步资产安全生命周期时序图

借助 C++ 的 RAII(资源获取即初始化)特性，渲染组件统一通过持有 `ResourceRef` 对象来管理资源生命周期。当某资源的最后一个引用被析构时，系统不会立刻触发硬件 API 进行显存释放，而是将释放操作推入帧感知销毁队列。该机制将实际的显存释放推移至 GPU 处理完当前帧指令之后执行，从而实现了安全的自动化显存回收管理。

5.2.3 光源预处理：蒙特卡洛积分 CDF 硬件同步

在基于物理的渲染与光线追踪管线中，计算大面积发光物体（Emissive Meshes）的软阴影通常采用下一事件估计（Next Event Estimation, NEE）重要性采样算法。由于纯随机采样会导致较高的蒙特卡洛方差，系统需要对场景中所有发光体的能量贡献进行预评估。

本模块中的 `LightManager` 组件承担了光源特征参数预处理与 GPU 同步的职责。

。

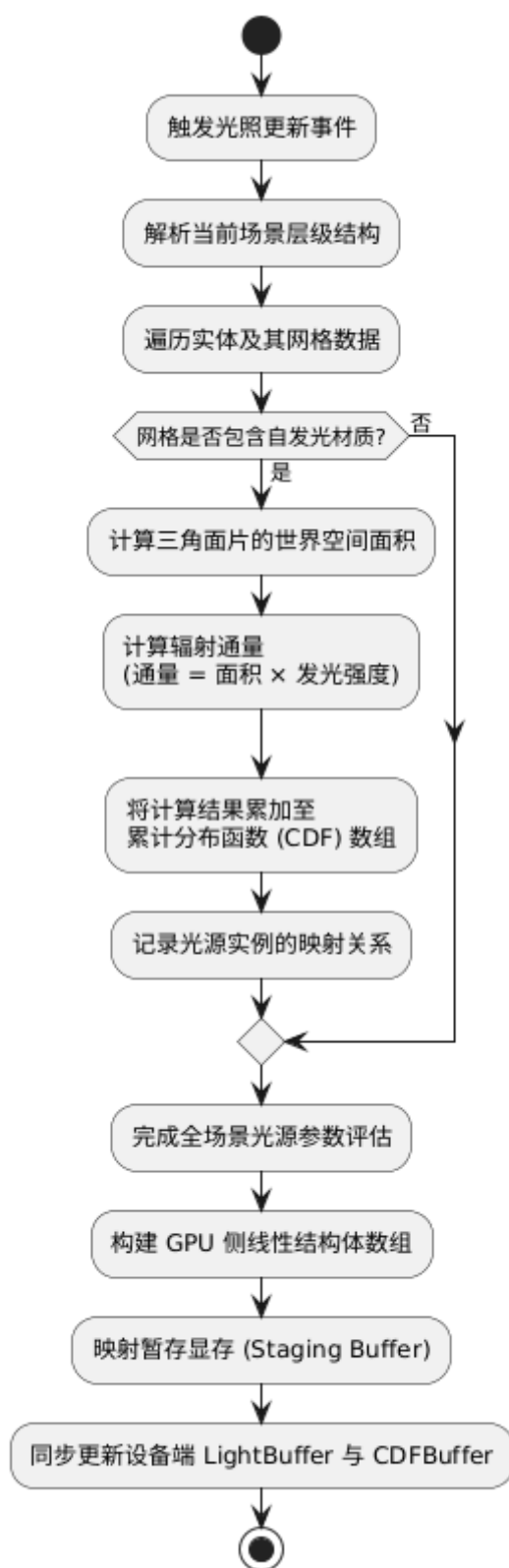


图 5.6 场景自发光体解析与 CDF 预建活动图

通过此活动流程，引擎在 CPU 端预先计算了场景中各发光三角面的概率密度分布，并构建了一维的累积分布函数（CDF）数组。将该数组同步至 GPU 后，光线追踪管线中的生成着色器（Ray Generation Shader）可通过二分查找（Binary Search）高效地根据物理能量权重选择采样光源。该前置计算步骤有效降低了光线追踪采样的初始方差，从而减轻了后续 SVGF 降噪阶段的滤波压力。

5.3 场景管理模块

场景管理模块是连接静态底层资产与动态渲染管线的核心枢纽。在混合渲染架构中，场景不仅需要处理传统的光栅化视锥体剔除（Frustum Culling），更需要实时维护光线追踪管线所依赖的三维空间拓扑（TLAS/BLAS）。同时，摄像机系统作为场景的观察者，必须为时域抗锯齿（TAA）和运动矢量（Motion Vectors）的计算提供高精度的矩阵投影与抖动状态。

5.3.1 混合光追加速结构(TLAS/BLAS)动态构建机制

硬件光线追踪无法直接遍历传统的顶点与索引缓冲。为了在极短的算力预算内完成光线与海量三角面片的求交测试，必须依托于 GPU 专用的双层边界体积层次结构，即底层加速结构（BLAS）与顶层加速结构（TLAS）。

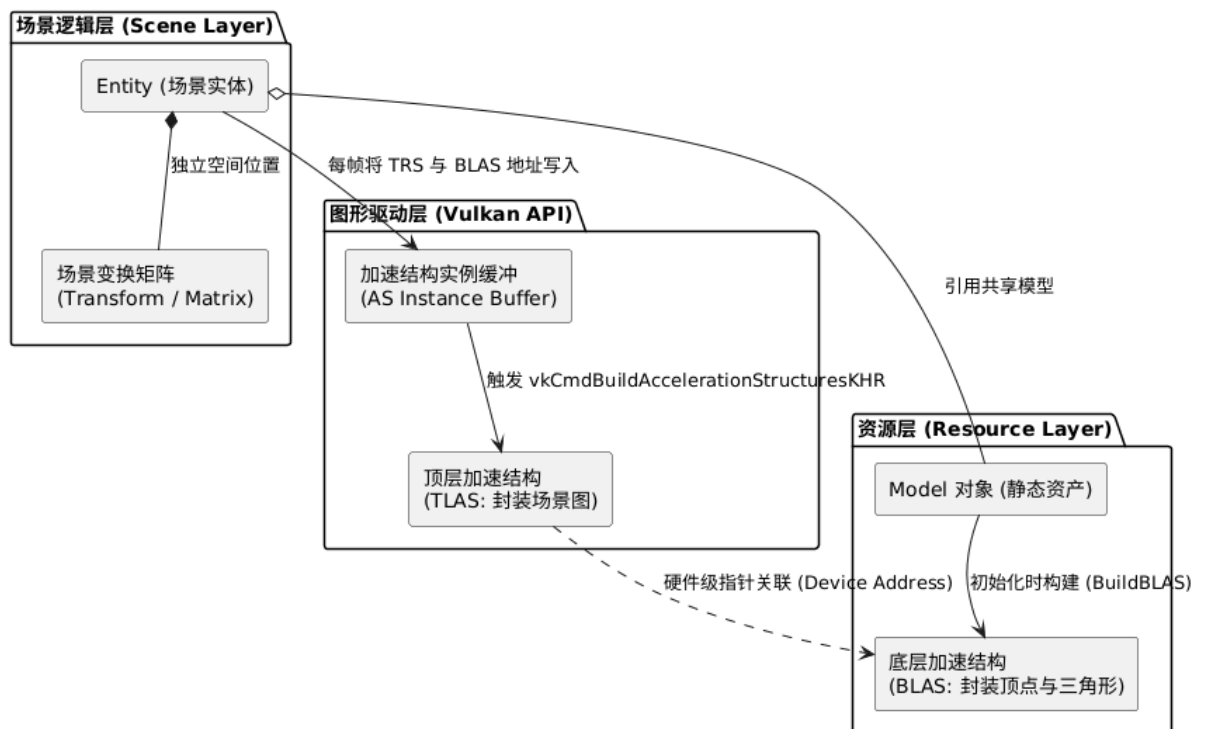


图 5.7 混合光追双层加速结构(AS)分级构建架构图

在该机制中，Model 模块负责预编译 BLAS（底层加速结构）。由于模型拓扑结构通常是静态的，BLAS 仅在资产加载阶段利用暂存缓冲（Scratch Buffer）在 GPU 端构建一次。而在主渲染循环中，Scene 模块负责监控所有 Entity 的空间变换矩阵（Transform）。一旦实体发生位移、旋转或缩放，场景将收集其关联的 BLAS 硬件地址与新的变换矩阵，重构实例缓冲（Instance Buffer），并在每帧提交前动态更新 TLAS（顶层加速结构）。这一机制确保了光线探测单元（RT Core）始终拥有绝对一致的物理世界视图，且避免了冗余的底层几何体重建开销。

5.3.2 基于八叉树的视锥体剔除策略

为了极大降低第一阶段 G-Buffer 提取时的光栅化压力，系统在 CPU 端实现了一套基于八叉树（Octree）的空间剔除方案。

针对场景中存在的大量静态与动态实体，Scene 对象会在空间初始化时构建八叉树节点网络。每个节点维护其覆盖的三维轴向包围盒（AABB）。在渲染流水线发起 GetVisibleEntities 查询时，系统采用深度优先遍历策略：将摄像机的视锥体（Frustum）与八叉树节点的 AABB 进行相交测试。若视锥体完全处于某节点外部，则该节点及其所有子节点下的 MeshComponent 将被直接抛弃，不再进入后续的 Vulkan 渲染指令录制（DrawIndexed）阶段。此举在 CPU 端过滤了海量不可见几何体，有效防止了 GPU 前端顶点处理单元的拥堵。

5.3.3 时域抖动注入与历史矩阵追踪系统

现代混合渲染引擎高度依赖时间维度的数据复用。无论是时空降噪（SVGF）还是时域抗锯齿（TAA），其核心基础均是精确的“屏幕空间运动矢量（Motion Vectors）”。本系统对 EditorCamera 进行了深度定制，引入了反向 Z（Reversed-Z）投影与亚像素抖动（Jitter）注入机制。

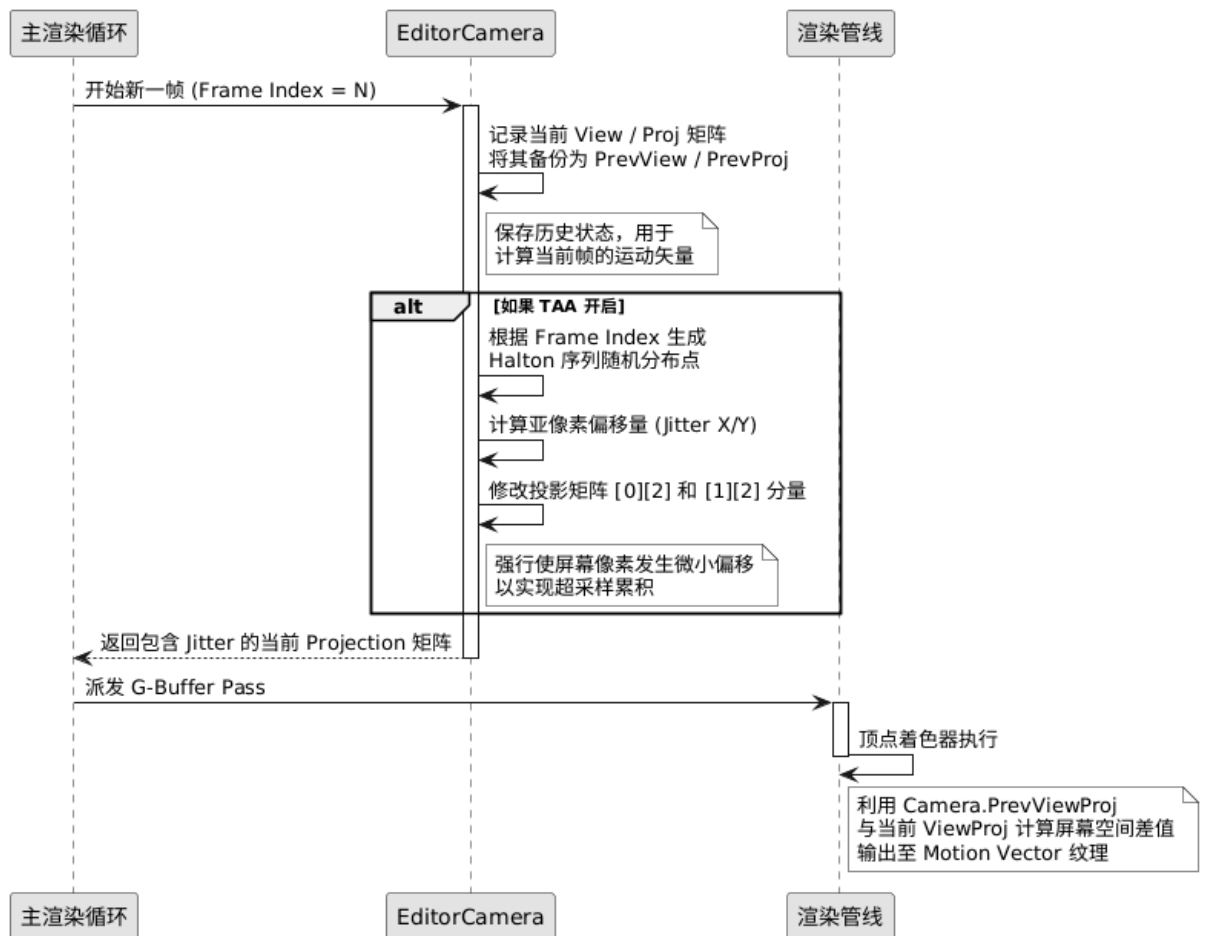


图 5.8 摄像机时域抖动注入与历史矩阵追踪时序图

1. 反向 Z 投影 (Reversed-Z): 由于浮点数在接近 0 时具有更高的精度, 相机类在构建透视投影矩阵时, 将远裁剪面 (Far Clip) 映射至 0.0, 近裁剪面 (Near Clip) 映射至 1.0。这一设计彻底消除了大尺度开放场景下的深度冲突 (Z-Fighting) 现象。
2. 亚像素抖动与速度缓冲: 当 TAA 激活时, 相机会利用 Halton 序列生成分布均匀的二维偏移量, 并动态注入到透视投影矩阵的第三列分量中。同时, 相机必须严格记录上一帧的视图投影矩阵 (PrevView 与 PrevProj)。在 G-Buffer 的顶点着色器阶段, 顶点会同时通过当前帧矩阵与历史帧矩阵进行变换, 两者的屏幕空间坐标差值即为真实的运动矢量 (Motion Vector)。这是指引 SVGF 降噪算法正确“回溯历史”的唯一数学依据。

5.4 渲染调度模块

渲染调度模块是引擎驱动复杂混合管线的核心中枢。由于本项目涉及光栅化、硬件

光线追踪及多级计算着色器（Compute Shader）的交替执行，各阶段间的资源读写依赖极其复杂。若采用传统的硬编码同步方案，不仅开发效率低下，且极易引发竞态条件（Race Condition）。

针对此问题，本模块实现了基于有向无环图（DAG）的渲染图（RenderGraph）系统。该系统通过声明式的资源依赖描述，实现了同步屏障的自动推导、资源生命周期的智能管理以及异构计算任务的统一调度。

5.4.1 任务编排与资源依赖的声明式建模

RenderGraph 的核心思想是将渲染过程抽象为一系列渲染通道（Pass）与资源（Resource）的组合。本系统通过 PassBuilder 接口，强制要求每个渲染通道在执行前声明其对资源的读写意图。

以下为渲染调度模块的组件交互与依赖建模设计：

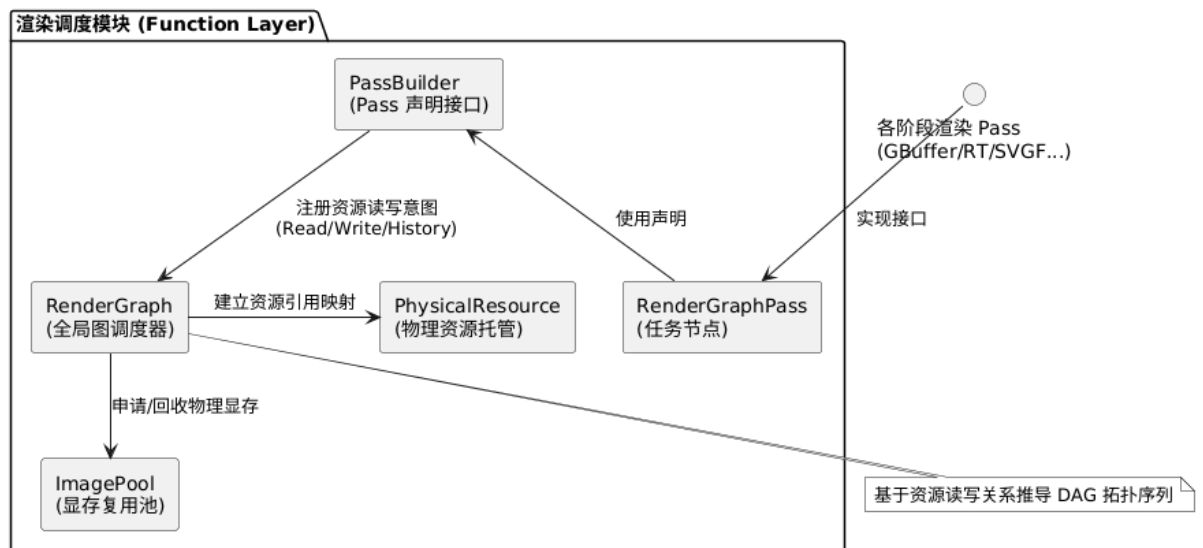


图 5.9 RenderGraph 资源依赖与任务编排组件设计图

5.4.2 自动同步屏障推导与状态转换机制

Vulkan 1.3 引入了 Synchronization2 扩展，极大简化了同步原语。本模块充分利用这一特性，在 RenderGraph 内部实现了一套自动化的资源状态自动机。当一个 Pass 声明对某资源进行“写入（Write）”而后续 Pass 声明“读取（Read）”时，系统会自动在指令流中插入精确的内存屏障。

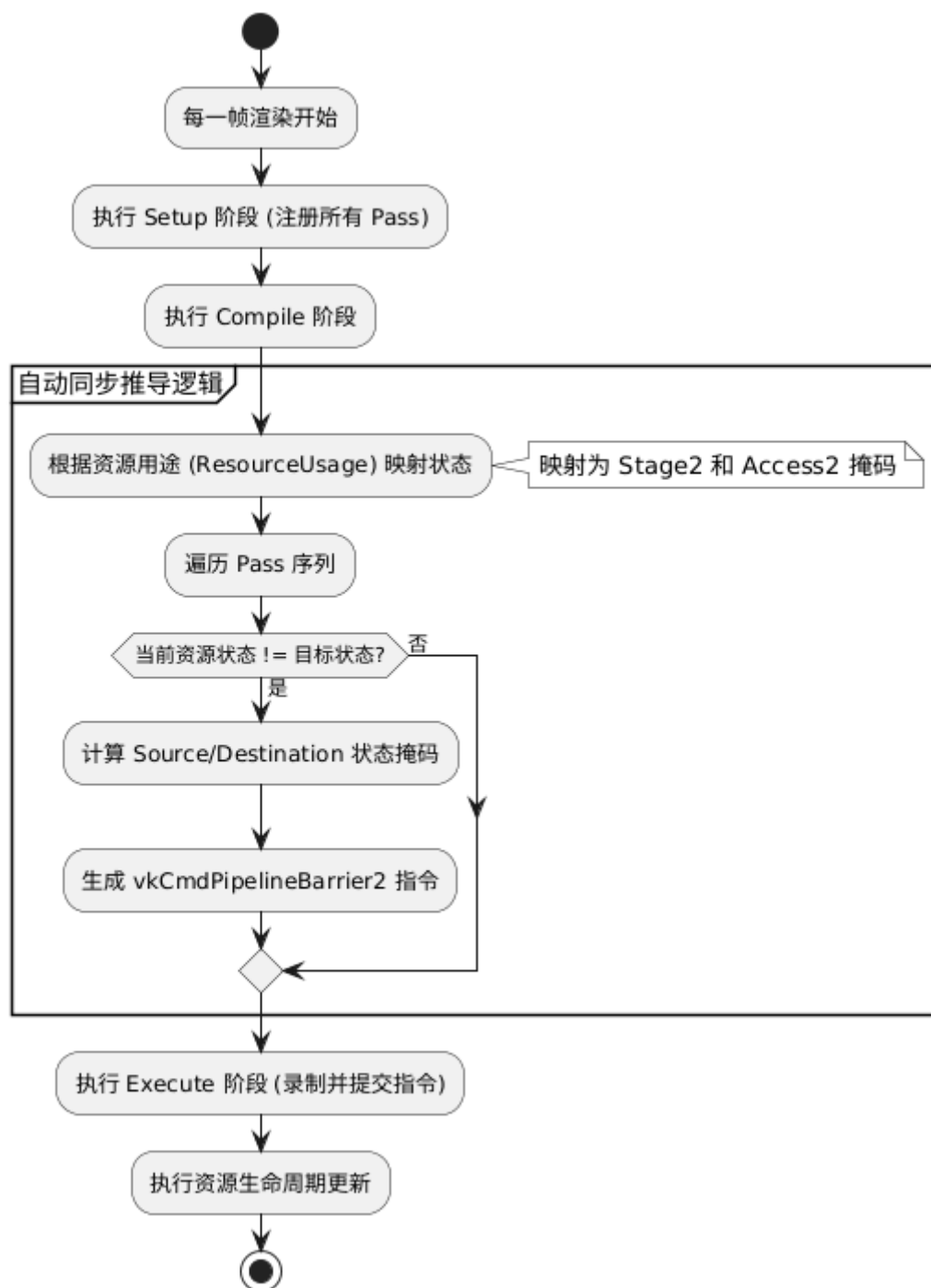


图 5.10 资源状态自动机与同步屏障自动生成活动图

该机制将复杂的 `VkPipelineStageFlags2` 和 `VkAccessFlags2` 掩码计算逻辑封装在调度层内部。例如，当光线追踪阶段 (Ray Tracing Stage) 输出的阴影图需要进入 SVGF 降噪阶段 (Compute Stage) 作为采样输入时，系统会根据预设的 `GetStateFromUsage` 逻辑，自动推导出从 `SHADER_STORAGE_WRITE` 到 `SHADER_READ_ONLY` 的布局转

换及对应的缓存刷新操作，确保了 GPU 端数据的绝对可见性。

5.4.3 异构计算任务的执行上下文封装

为了提升渲染逻辑的可读性并降低 Vulkan 描述符集（Descriptor Sets）管理的复杂度，系统针对不同类型的硬件计算任务设计了统一的执行上下文（Execution Context）体系。该体系通过对底层指令缓冲（CommandBuffer）的二次封装，为光栅化、通用计算及硬件光线追踪提供了领域专用接口。

在具体设计上，系统实现了接口的专业化与抽象化。针对不同的管线需求，引擎提供了定制化的上下文实现：例如，GraphicsExecutionContext 封装了视口设置、顶点缓冲绑定及光栅化流水线绑定等图形接口；ComputeExecutionContext 专注于计算核函数的派发（Dispatch）；而 RaytracingExecutionContext 则负责空间加速结构的绑定与光线追踪指令（TraceRays）的下发。这种层次划分使得具体的算法实现能够与底层的 Vulkan 原生 API 调用相解耦，从而提升了业务代码的可维护性。

同时，为简化描述符管理流程，该体系集成了着色器反射（SPIR-V Reflect）数据，实现了资源绑定的自动化。在渲染通道（Pass）执行期间，上下文会动态解析当前着色器的资源布局信息，完成描述符集的更新与绑定工作。开发者只需通过抽象的资源句柄指定管线的输入输出，上下文模块便会在后台自动处理绑定偏置（Binding Offsets）与描述符池的分配逻辑。此外，为实现异构任务的统一调度，所有类型的执行上下文均继承自同一基类，共享一致的推入常量（Push Constants）下发机制与底层指令缓冲管理逻辑。这种统一的底层抽象确保了光栅化与光线追踪任务在 RenderGraph 调度器中具备标准化的接口规范，从而支持了跨阶段渲染管线的协同执行与计算资源的集中调度。

5.5 混合渲染管线模块

在完成了底层的资源调度与空间结构管理后，本节将重点论述混合渲染管线（Hybrid Rendering Pipeline）的核心算法实现。混合管线旨在结合光栅化（Rasterization）的高效几何处理能力与硬件光线追踪（Hardware Ray Tracing）的物理精确性。系统将整体算法拆解为几何特征提取、低采样率光影探测以及时空降噪滤波三个依次递进的渲染通道（Pass）。

5.5.1 基于光栅化的几何特征提取（G-Buffer）

混合管线的首个阶段是剥离场景的几何与材质信息，避免在后续昂贵的光线追踪阶

段发生无意义的过度绘制（Overdraw）。本系统利用光栅化管线提取屏幕空间的可见表面数据，并将其压缩存储至多张渲染目标(Render Targets)中，构成几何缓冲(G-Buffer)。

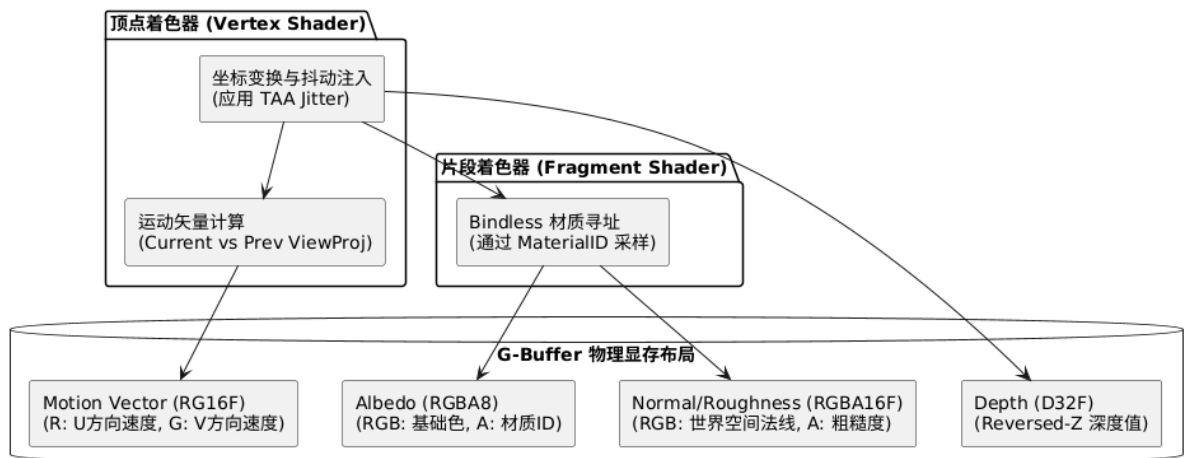


图 5.11 几何特征提取阶段与 G-Buffer 显存布局图

在此阶段，系统为支持时域抗锯齿（TAA）与时空降噪（SVGF），在顶点着色器中引入了历史矩阵追踪机制。顶点不仅通过当前帧的视图投影矩阵计算屏幕坐标，同时利用上一帧的历史矩阵计算前置屏幕坐标。两者的屏幕空间坐标差值被量化并输出至运动矢量纹理（Motion Vector Texture）中。此外，法线数据被映射至高精度的 RGBA16F 格式以防止量化误差导致的光线反射偏移，而深度缓冲则采用反向 Z（Reversed-Z）映射以保障远距离视角的深度测试精度。

5.5.2 单样本（1-SPP）光线追踪探测

在获取了屏幕空间的几何属性后，管线进入光线追踪探测阶段。受限于实时渲染的毫秒级算力预算，系统无法采用离线渲染中高频率的蒙特卡洛积分（Monte Carlo Integration）。本模块实现了基于每像素单次采样（1-SPP）的硬件光追策略，主要用于计算精确的软阴影与环境光遮蔽（AO）。

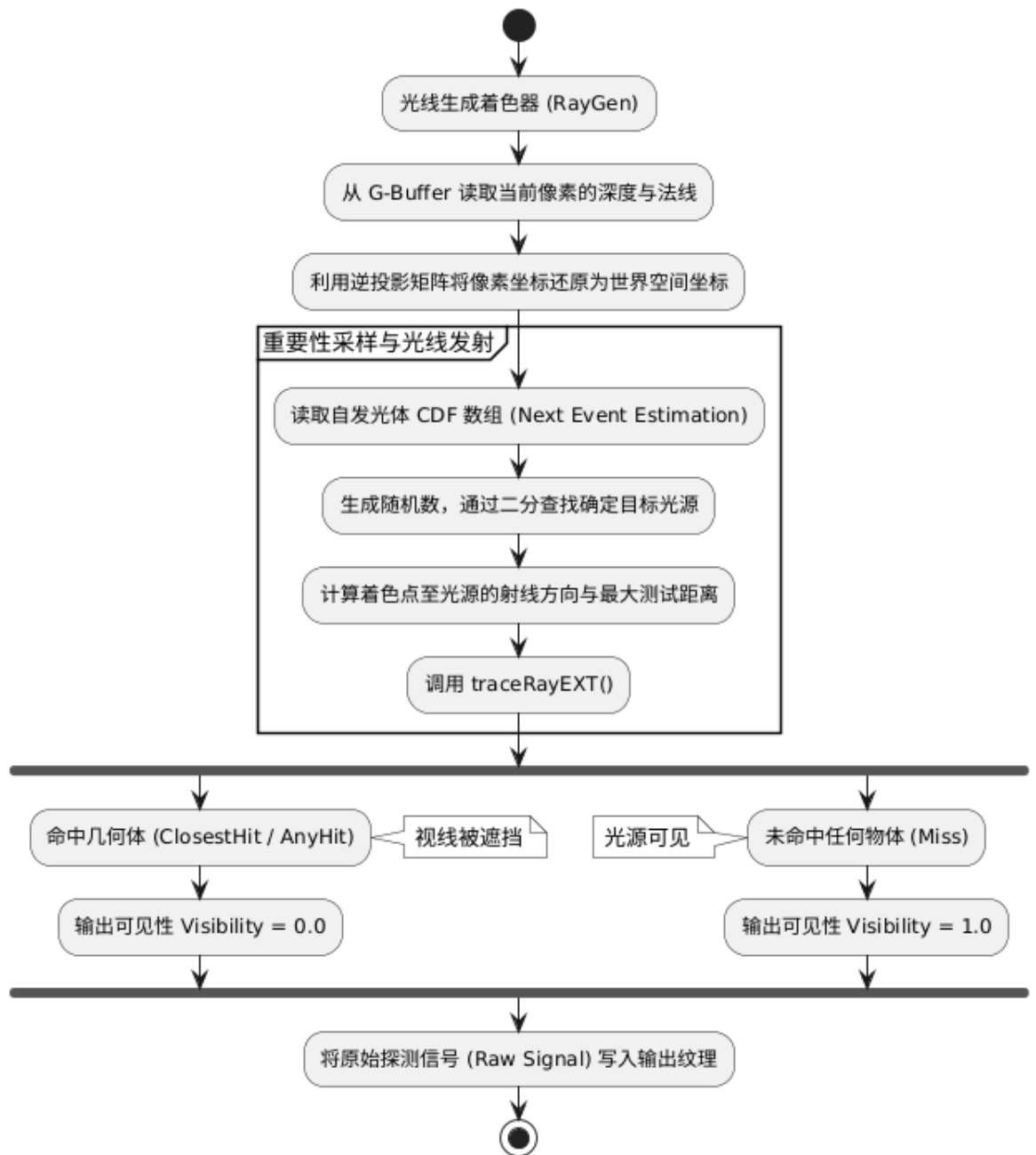


图 5.12 1-SPP 硬件光线追踪软阴影探测活动图

在光线生成（RayGen）着色器中，算法首先根据 G-Buffer 还原像素的三维世界空间坐标。为了降低 1-SPP 带来的严重方差（噪点），系统利用了 5.2 节构建的光源累积分布函数（CDF）进行下一事件估计（NEE）。算法倾向于向能量贡献度更高的自发光体发射测试光线，从而在极低的采样率下获取更加可靠的初始光影信号。探测结果（被遮挡或可见）以原始噪声纹理的形式输出，作为降噪器的输入先验。

5.5.3 时空方差引导滤波（SVGF）降噪实现

由于 1-SPP 探测输出的信号包含剧烈的高频噪声，无法直接用于最终画面合成。本模块引入了时空方差引导滤波（Spatiotemporal Variance-Guided Filtering, SVGF）算法。该算法通过时间维度的历史累积与空间维度的自适应模糊，重构出纯净的物理光影。

算法的执行流程被拆分为三个顺序执行的计算着色器（Compute Shader）通道：

1. 时域重投影与累积 (Temporal Reprojection): 算法首先利用几何阶段输出的运动矢量（Motion Vector），在历史帧缓冲中查找当前像素的前置状态。系统通过比较当前帧与历史帧的深度差与法线夹角来判定历史数据的有效性。若判定有效，则根据指数滑动平均（EMA）模型混合当前帧信号与历史帧信号；若判定为几何遮挡或场景切换导致的历史失效，则丢弃历史数据，将累积历史帧数重置为 0。
2. 空间方差估计 (Variance Estimation): 时间累积能够有效降低静态区域的噪声，但在运动物体边缘或新出现的区域，因历史数据重置，局部方差依然极高。此通道通过计算像素周围 7×7 邻域内的光照一阶矩与二阶矩，估算出当前像素的统计学方差。该方差将作为后续空间滤波器的引导权重。
3. 自适应 A-Trous 空间滤波 (A-Trous Spatial Filtering): 针对高方差区域，系统采用多级 A-Trous 小波变换进行空间模糊。为了在抹平噪声的同时保留尖锐的几何边缘与阴影边界，滤波器引入了基于 G-Buffer 属性的联合双边过滤权重（Edge-stopping Function）。其权重函数 w 综合了深度（Depth）、法线（Normal）与光照亮度（Luminance）的差异：

$$w = \exp \left(-\frac{|\Delta z|}{\sigma_z} - \frac{\arccos(N_1 \cdot N_2)}{\sigma_n} - \frac{|\Delta L|}{\sigma_L \sqrt{V}} \right) \quad (5-1)$$

其中 Δz 与 N 分别代表深度差与法线， ΔL 代表亮度差， V 为上一步估算的方差。方差 V 越大，亮度权重项的惩罚越小，模糊程度越深。通过多次迭代不同步长的 A-Trous 滤波，算法最终以较低的性能开销实现了大范围的噪声平滑。

在经过上述几何采集、物理探测与时空降噪后，纯净的光影信号将与基础颜色（Albedo）进行 PBR 物理合成，最终进入后期处理管线进行色调映射（Tone Mapping）与输出。

5.5.4 光照合成与后期处理管线

在时空降噪（SVGF）阶段完成后，管线获取了无噪点的纯净物理光影信号。然而，此时的数据仍处于高动态范围（HDR）的线性色彩空间中，且缺乏材质的基础表面属性。为了输出最终的设备可视图像，系统设计了光照合成与后期处理管线，该管线由三个依次递进的着色器阶段构成。

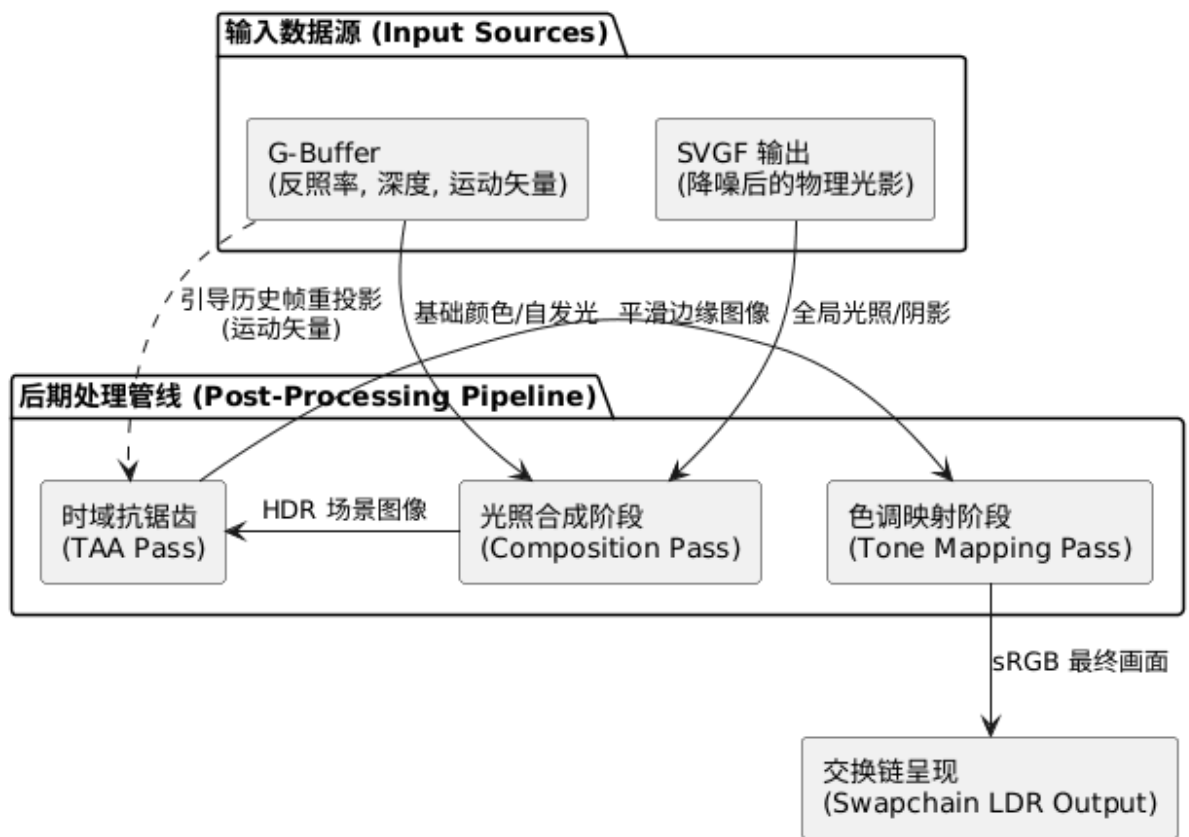


图 5.13 光照合成与后期处理数据流转图

1. 物理光照合成 (Composition): 合成着色器首先从 G-Buffer 中采样反照率 (Albedo) 与自发光 (Emission) 参数, 并将其与 SVGF 输出的漫反射 (Diffuse) 及镜面反射 (Specular) 光影进行数学乘加运算。在此阶段, 系统还会根据环境映射参数融合天空盒 (Skybox) 的背景辐射度, 从而还原出真实的物理材质质感。
2. 时域抗锯齿 (Temporal Anti-Aliasing, TAA): 虽然 SVGF 解决了蒙特卡洛积分的方差噪点, 但光栅化阶段产生的几何边缘阶梯状锯齿依然存在。TAA 通道利用几何提取阶段注入的亚像素抖动 (Jitter), 结合运动矢量 (Motion Vectors) 对历史帧进行重投影累积。算法通过对当前帧邻域像素的色彩空间 (如 YCoCg 空间) 进行方差限制 (Variance Clipping), 有效抑制了历史数据重投影时常见的“鬼影 (Ghosting)”伪像, 实现了极低开销的高质量边缘平滑。
3. 色调映射与伽马校正 (Tone Mapping & Gamma): 由于 HDR 图像的亮度分布远超常规显示设备的色域上限, 最终的后期处理通道引入了色调映射算法 (如 ACES 拟合曲线)。该算法对极高亮度的像素进行平滑的高光压制 (Highlight Compression), 防止画面过曝丢失细节; 随后执行标准的伽马校正 (Gamma

Correction），将线性空间的物理光照转换为 sRGB 色彩空间，最终交由图形界面的交换链（Swapchain）进行屏幕呈现。

此阶段的执行标志着单帧渲染管线数据流转的彻底结束。

5.6 交互调试模块

在实时渲染引擎的开发与优化过程中，硬编码（Hard-coding）参数往往会导致极低的迭代效率。为了向图形算法开发者提供非侵入式的分析环境，本系统在渲染管线之上构建了交互调试与工具层模块。该模块基于层叠式架构（Layer Stack）与即时模式图形界面（Dear ImGui），实现了应用生命周期的统一托管、渲染参数的闭环调优以及底层 GPU 性能的可视化侦测。

5.6.1 应用生命周期与事件驱动机制

为了提升开发效率，系统实现了一套基于事件驱动的参数调节机制，在该数据反馈闭环中，引擎的应用层以 Application 类为核心枢纽，负责接管操作系统的窗口事件、统筹各个功能层（Layer）的更新，并驱动渲染主循环。为实现业务逻辑与底层图形后端的解耦，系统采用了基于层级栈的事件路由机制。

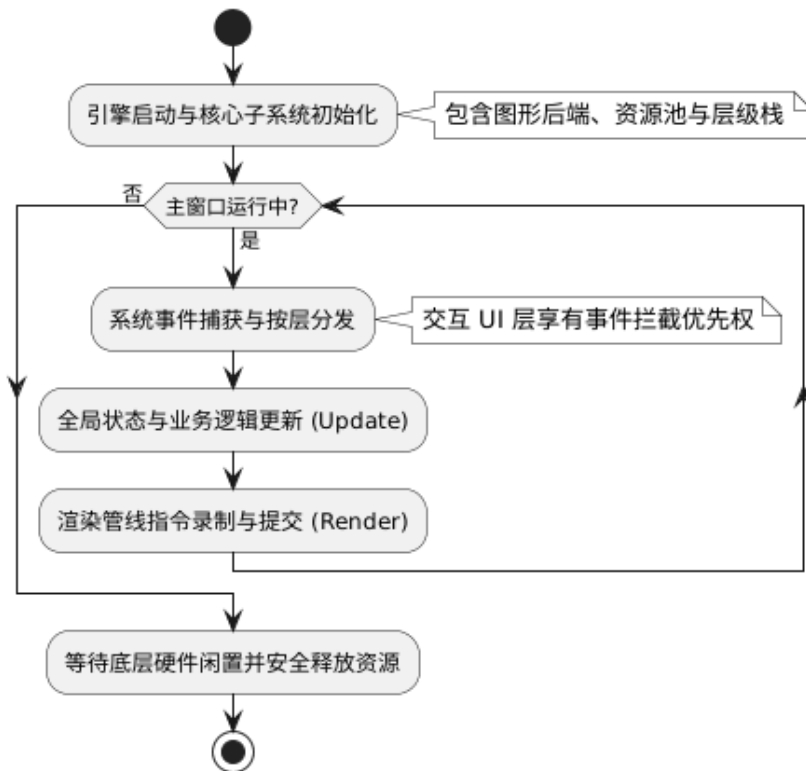


图 5.13 渲染参数动态调节与 UBO 更新顺序图

在该机制下，用户的键鼠输入首先由操作系统捕获并转化为标准事件，随后按层级栈从顶至底传递。当用户在交互界面（如控制面板）上操作时，`ImGuiLayer` 会拦截该事件，防止其向下穿透触发场景相机的误操作。这种设计保证了调试界面与场景漫游逻辑的互不干扰。

5.6.2 非侵入式参数反控与闭环调优

混合渲染管线中的诸多参数（如曝光度、降噪器的时空权重、采样率等）直接影响最终的画面质量与执行效率。为实现非侵入式的参数调优，系统建立了从前端 UI 到 GPU 显存的数据同步通路。

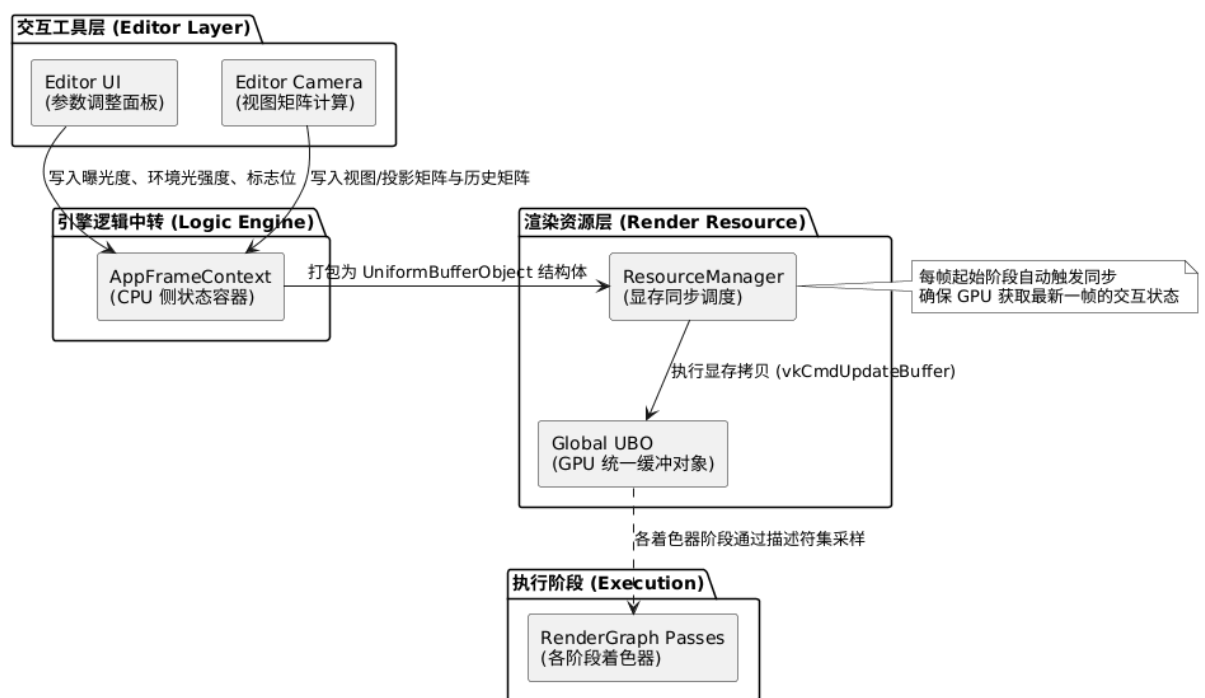


图 5.14 渲染参数实时控制与显存同步数据流向图

在该机制下，交互层产生的参数变更首先被记录在 CPU 端的 `AppFrameContext` 状态容器中。在每帧的更新阶段，系统通过 `ResourceManager` 将这些散列的状态量打包成符合显存对齐规范的 `UniformBufferObject` 结构体，并利用 Vulkan 的显存拷贝指令同步至全局统一缓冲 (UBO)。这种设计实现了交互数据与渲染管线的解耦，支持在不重新编译着色器的情况下实现渲染效果的即时反馈。

5.6.3 图形界面后端与性能可视化

为了支持对复杂渲染管线的深度审查，系统实现了针对 Vulkan 后端的图形界面适

配，并集成了性能侦测模块。

在资源展示层面，交互层通过独立分配的描述符池，实现了渲染目标的动态挂载。开发者可以将 `RenderGraph` 在各中间阶段生成的图像（如 `G-Buffer` 的法线图、光线追踪产生的原始阴影图等）直接映射至 UI 窗口中进行可视化审查。这种能力对于定位混合管线中的数据冒险或算法逻辑错误具有重要作用。

在性能监控层面，系统利用硬件提供的时间戳查询（`Timestamp Queries`）机制，在渲染任务图的每个通道起点与终点插入埋点。系统采集这些微秒级的耗时数据，并在交互面板中以图表形式实时展示各渲染阶段的 GPU 算力分布。这为衡量降噪算法的开销、平衡光线追踪采样率与实时性之间的关系提供了客观的数据依据，支持了渲染管线的针对性优化。

5.7 本章小结

本章完成了系统六大核心模块的底层逻辑设计。通过对调度算法、资源模型及信号重建链路（`SVGF/TAA`）的深度拆解，形成了严密的详细设计方案，为下一章的系统实现奠定了坚实的工程基础。

第六章 系统实现与测试

在完成了混合渲染引擎的宏观架构设计与核心模块的详细编码实现后，本章将对系统全面的运行测试与功能验证。本章首先明确了系统编译与运行的软硬件环境，随后通过截图直观展示了混合管线各阶段的渲染成果，最后通过设计严谨的系统测试用例，验证了各核心功能模块的鲁棒性与正确性。

6.1 系统运行环境

操作系统：Windows 11；

开发软件：Visual Studio Code；

软件环境：C++ 20 Vulkan SDK (兼容 1.3/1.4 规范)；

图形处理器 (GPU)：NVIDIA GeForce RTX 5070 Ti Laptop GPU (12GB VRAM)

6.2 系统功能实现

本节将通过引擎的实际运行画面，按数据流转的顺序，通过截取并分析从前端交互界面到管线各个中间阶段（如几何提取、光追探测、降噪滤波等）的物理渲染输出结果，直观地论证前文所述各项核心算法在工程落地中的正确性与有效性。

6.2.1 交互式调试界面展示

系统集成了非侵入式的 ImGui 调试面板。面板左侧提供了场景实体列表与层级树，右侧展示了渲染参数反控接口。

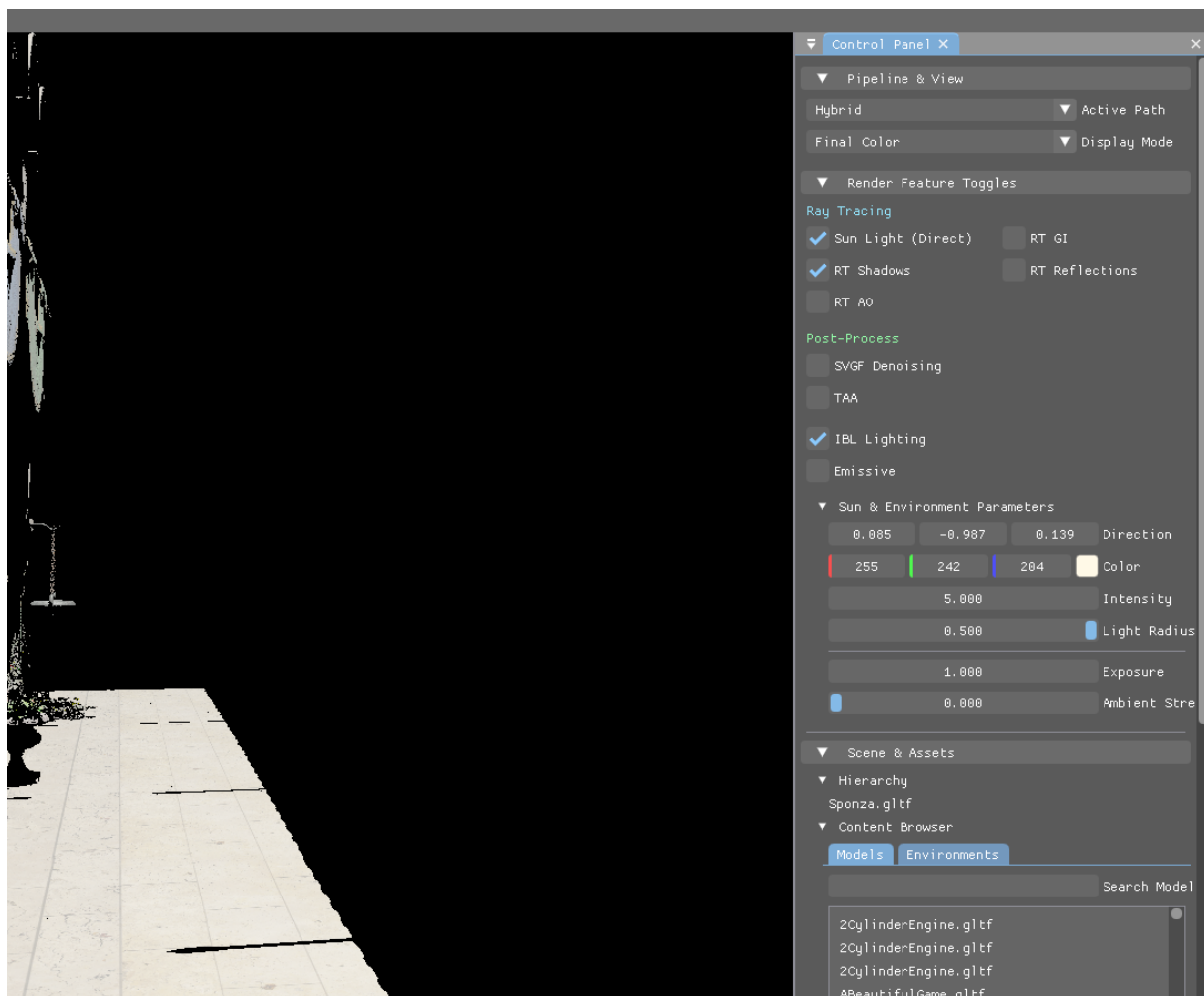


图 6.1 交互面板图

6.2.2 G-Buffer 几何特征可视化

几何特征提取是混合管线的基础。通过工具层的通道可视化功能，可以直接观测到第一阶段光栅化输出的各项物理属性。

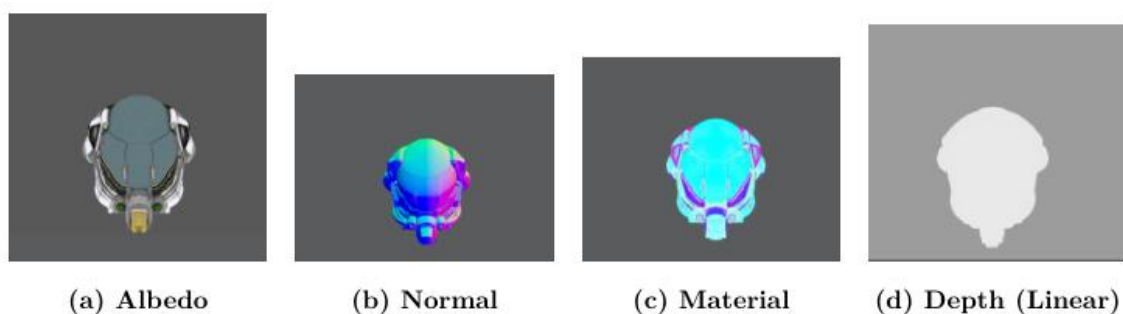


图 6.2 G-Buffer 多附件几何信号采集分量图

6.2.3 光线追踪与降噪算法对比

本小节展示引擎核心的物理光影解算能力。光线追踪 1-SPP 采样生成的原始阴影

包含剧烈噪声，经过 SVGF 降噪后得到了平滑的物理软阴影。



(a) 原始 1 SPP 间接光照 (高方差噪声)



(b) SVGF 滤波后结果 (平滑且保留细节)

图 6.3 SVGF 降噪前后的对比分析图

6.2.4 后期处理与最终画面输出

在光影合成后，画面经过时域抗锯齿（TAA）与色调映射（Tone Mapping）输出最终结果。



(a) TAA OFF



(b) TAA ON

图 6.4 时域抗锯齿 (TAA) 开启前后视觉效果对比图

6.3 系统测试

系统的高可用性与架构鲁棒性是评估渲染引擎工程落地价值的核心指标。为全面验证本文所构建的混合渲染引擎在复杂交互场景与动态数据流转下的执行稳定性，本节针对系统的底层核心模块（涵盖异步资源的生命周期调度、RenderGraph 的 DAG 拓扑推导、管线状态的热切换等）开展了系统级综合测试。

在整个测试生命周期内，系统借助 Vulkan 官方验证层（Validation Layers）与底层性能探测器，对显存分配水位、同步屏障状态及指令提交流程进行了严格的实时监控。这些测试不仅验证了单次功能调用的正确性，更从架构层面排除了长期运行下潜在的内存泄漏与设备丢失隐患具体的系统核心功能测试用例及验证结果如表 6.1 所示。

表 6.1 系统核心功能测试用例表

测试 编号	测 试 功能	操作步骤	预期结果	测 试结果
T01	资 产 异步加载	从 UI 加载 200MB 的 GLTF 场景	界面无卡顿，模型完整渲染	通 过
T02	渲 染 路径切换	在控制面板 中 连 续 切 换 Forward 与 Hybrid 渲染模式	RenderGraph 成功执行 DAG 拓扑重构，显存屏障合法，画面正确更新且无崩溃	通 过
T03	AO 独立控制	勾 选 Ray Traced AO 调节滑 块	画面即时产生柔和遮蔽感	通 过
T04	动 态 实体移除	选中模型并 点击 Remove	Deletion Queue 触发，系统 不崩溃	通 过
T05	调 试 预览输出	切换显示模 式至 Normal 分 量	屏幕输出法线编码颜色视 图	通 过

6.4 本章小结

本章对混合渲染引擎进行了系统的运行展示与测试验证。首先界定了软硬件基准环境；其次，结合实际渲染画面，验证了 G-Buffer 提取、光线追踪采样、SVGF 降噪及后期处理等核心算法的正确性；最后，通过系统级测试用例，确认了资源异步加载与显存调度架构的执行稳定性。测试结果表明，本引擎达到了预期设计目标，能够支撑现代高性能图形渲染的开发需求。

第七章 总结与展望

7.1 全文工作总结

本文针对现代实时渲染中光线追踪计算开销大、低采样率下噪声泛滥的业界痛点，基于 Vulkan 1.3 图形 API 从零构建了一套高性能的实时混合渲染引擎。本文的核心工作与主要贡献总结如下：

1. 设计了基于无绑定架构与 RenderGraph 的自动化调度中枢：系统在资源层实现了全局无绑定（Bindless）架构与基于 RAII 的显存生命周期托管，消除了传统的逐对象绑定开销。在调度层，设计并实现了数据驱动的 RenderGraph 架构，利用有向无环图（DAG）实现了渲染任务的逻辑解耦；通过自动化推导资源状态生成精确的同步屏障（Barrier），彻底解决了 Vulkan 异构计算中手动同步繁琐且易错的问题，并实现了时域历史资源的持久化，为后续算法奠定了底层框架。

2. 构建了结合光源重要性采样的硬件混合光追管线：系统充分结合了光栅化 G-Buffer 特征提取的高效性与硬件光线追踪（RT Pipeline）的物理精确性。针对单次采样（1-SPP）极低采样率导致的方差过高的问题，系统在 CPU 端预先评估了场景自发光体的物理辐射度，构建了光源能量的累计分布函数（CDF）。在光线探测阶段，系统引入了基于光源能量分布的重要性采样（Importance Sampling），有效引导射线向高贡献度光源发射，极大地抑制了原始物理探测信号的初始噪点，显著提升了场景软阴影与全局光影的收敛速度。

3. 落地了高性能的时空信号重建与后期处理系统：深入研究并实现了时空方差引导滤波（SVGF）算法，在极低采样率下通过历史重投影累积与自适应的联合双边 $\hat{A}$ -Trous 空间滤波，实现了高质量的光影降噪。此外，系统集成了时域抗锯齿（TAA）与 ACES 色调映射后期处理方案，通过亚像素抖动与色彩空间方差裁剪技术，消除了几何边缘走样并还原了极具质感的 PBR 物理画质。

7.2 研究不足与未来展望

尽管本文构建的混合渲染系统已具备较高的工程完备性并达到了预期的实时渲染指标，但在算法深度与功能维度的覆盖上仍有进一步提升的空间。未来的研究与优化工作可围绕以下方向展开：

1. 多弹射全局光照（Multi-bounce GI）与高级采样算法：目前的系统主要聚焦于直接光软阴影与单次弹射的间接光照。未来可考虑引入储层时空重要性重采样（ReSTIR）算法，通过时域与空域的样本重用（Sample Reuse），在有限的算力预算内实现逼真且稳定的多弹射全局光照（Global Illumination）与焦散（Caustics）效果。

2. 复杂微表面物理材质的扩展支持：当前引擎主要支持标准的 Cook-Torrance PBR 材质模型。未来可进一步扩展底层着色器机制，引入对各向异性（Anisotropy）、次表面散射（SSS，适用于皮肤与玉石材质）、清漆层（Clearcoat）以及薄透镜折射等高级材质特性的硬件光追支持。

3. 深度学习超分辨率与光追重建技术的集成：虽然 SVGF 与 TAA 的结合解决了大部分噪声与锯齿问题，但在极端动态场景下仍可能出现历史重投影带来的模糊或鬼影（Ghosting）现象。未来研究可考虑集成 NVIDIA DLSS 3.5 (Ray Reconstruction) 或 AMD FSR 3 等前沿技术，利用 AI 神经网络辅助特征提取与信号重建，这将是进一步提升画面的核心方向。

4. GPU 驱动渲染与显存物理别名化深度优化：在渲染管线的性能的深度挖掘方面，目前的 RenderGraph 虽实现了逻辑解耦，但尚未完全挖掘显存物理复用（Aliasing）的极限潜力。未来可引入基于资源生存区间段的严格重叠分配策略以进一步压低显存峰值。同时，可探索 Vulkan 1.3 的网格着色器（Mesh Shaders），将 CPU 端的八叉树剔除全面下放至 GPU 执行，实现真正的纯 GPU 驱动渲染架构。

参考文献

- [1] Akenine-Möller T, Haines E, Hoffman N. Real-Time Rendering[M]. 4th ed., Boca Raton: CRC Press, 2018, 1-10, 760-785.
- [2] Microsoft. Announcing Microsoft DirectX Raytracing[Z]. Redmond: Microsoft Blog, 2018.
- [3] Khronos Group. Vulkan 1.3 Core API Specification[S]. Beaverton: Khronos Group, 2022.
- [4] Gregory J. Game Engine Architecture[M]. 3rd ed., Boca Raton: CRC Press, 2018, 20-55.
- [5] O'Donnell Y. FrameGraph: Extensible rendering architecture in Frostbite[C]. SIGGRAPH 2017 Courses, ACM, Los Angeles, 2017, 1-8.
- [6] Schied C, Kaplanyan A, Wyman C, et al. Spatiotemporal variance-guided filtering: real-time reconstruction for path-traced global illumination[C]. Proceedings of High Performance Graphics, ACM, Los Angeles, 2017, 1-10.
- [7] Pharr M, Jakob W, Humphreys G. Physically Based Rendering: From Theory to Implementation[M]. 3rd ed., Cambridge: Morgan Kaufmann, 2016, 110-145, 520-530.
- [8] Cook R L, Torrance K E. A reflectance model for computer graphics[J]. ACM Transactions on Graphics (TOG), 1982, 1(1): 7-24.
- [9] Walter B, Marschner S R, Li H, et al. Microfacet models for refraction through rough surfaces[C]. Proceedings of the 18th Eurographics conference on Rendering Techniques, Eurographics Association, Aire-la-Ville, 2007, 195-206.
- [10] Schlick C. An inexpensive BRDF model for physically-based rendering[J]. Computer Graphics Forum, 1994, 13(3): 233-246.
- [11] Sellers G, Kessenich J. Vulkan Programming Guide: The Official Guide to Learning Vulkan[M]. 1st ed., Boston: Addison-Wesley Professional, 2016, 35-80.
- [12] Haines E, Akenine-Möller T. Ray Tracing Gems: High-Quality and Real-Time Rendering with DXR and Other APIs[M]. Berkeley: Apress, 2019, 150-180.
- [13] Marrs C, Shirley P, Wald I. Ray Tracing Gems II: Next Generation Real-Time Rendering with DXR, Vulkan, and OptiX[M]. Berkeley: Apress, 2021, 200-230.
- [14] Reed N. Depth precision visualized[Z]. NVIDIA Developer Blog, 2015.

- [15] Salvi M. An excursion in temporal supersampling[C]. Game Developers Conference, GDC, San Francisco, 2016, 1-10.
- [16] Cornut O. Dear ImGui: Bloat-free graphical user interface for C++ with minimal dependencies[CP]. GitHub, 2014.

致谢

时光荏苒，本科学习生涯即将落幕，毕业设计的完成也为这段求学之旅画上圆满句号。从论文选题、框架搭建到最终定稿，每一步都离不开各位老师、家人与朋友的帮助，在此，我谨向所有给予我支持的人致以最诚挚的谢意。

首先衷心感谢我的指导老师韩强老师，老师以严谨的治学态度和耐心的指导为本文的撰写提供了诸多帮助。然后感谢学院各位授课老师，本科期间传授的专业知识与思维方法，为本次毕业设计奠定了坚实基础，也让我提升了专业素养与独立思考能力。也感谢我的家人，他们是最坚实的后盾，始终给予我无私的关爱与支持，为我创造安心的学习环境，他们的期盼是我前进的不竭动力。

最后，再次感谢所有在论文撰写中给予我帮助的人们。